RailFlow: A Modern Railway Reservation System (RRS)

Software Engineering Project Documentation

A Full-Stack, Role-Based, Concurrency-Safe Train Reservation Platform with Segment-Level Seat Inventory, Dual-Source Timetable Search, and Live Telemetry Simulation

React 19 | TypeScript | Vite 8 | Tailwind CSS
Node.js | Express 4.21 | Prisma 5.22 | Turso (libSQL) / SQLite
Netlify (Frontend CDN) | Render (Backend API)

Academic Year: 2026–2027
November 2026

Declaration

I hereby declare that this software engineering project report titled **"RailFlow — A Modern Railway Reservation System"** is an authentic record of our own work carried out as an academic portfolio and capstone project. The implementation utilizes modern web standards, an Express-based TypeScript application server, Prisma ORM, and the Turso distributed libSQL database. All external libraries, open-source datasets, and references have been cited appropriately.

Acknowledgements

We express our sincere gratitude to our academic mentors, faculty advisors, and software engineering peers who provided guidance, code review feedback, and technical support throughout the conception, architecture, and implementation of the RailFlow platform. We

also acknowledge the open-source software communities behind Node.js, Express, React, Prisma, Turso, Tailwind CSS, and the Indian Railway Open Data projects whose tools and datasets made this comprehensive technical showcase possible.

Abstract

Passenger railways represent one of the world's most intricate and heavily contended transaction domains. Thousands of passengers simultaneously compete for a finite inventory of physical berths across multi-stop transit corridors. Unlike conventional retail e-commerce inventory where a purchased unit is permanently decremented, a railway passenger berth can be partitioned and resold across non-overlapping geographical segments: the same physical seat may accommodate Passenger A from Station 1 to Station 3, and Passenger B from Station 3 to Station 6 without conflict.

RailFlow is an enterprise-grade, concurrency-safe, full-stack railway reservation platform developed to model the full lifecycle of passenger booking, inventory control, and operational transit management. The application is architected around a robust three-tier model: a high-performance **React 19 single-page application (SPA)** built with **Vite 8** and **Tailwind CSS**, hosted on **Netlify**; an **Express 4.21 TypeScript backend API server** hosted on **Render**; and a primary **Turso (libSQL/SQLite)** database managed through **Prisma ORM 5.22.0** utilizing official driver adapters (`@prisma/adapter-libsql`).

The system implements a **dual data-source architecture**: an internal high-concurrency relational master database providing bookable inventory with serializable ACID transactions, alongside a comprehensive in-memory catalogue of over 8,000 stations and scheduled Indian Railways timetables loaded at startup from curated open datasets. Key technical capabilities include:

1. Deterministic segment-level berth allocation preventing lost-update anomalies and double-booking under high concurrency.
2. An automatic background hold-cleanup engine that releases temporary seat locks upon a 10-minute time-to-live (TTL) expiration.
3. Accurate server-side fare calculation incorporating distance tiers, superfast surcharges, class multipliers, and statutory GST.
4. Two-phase simulated payment processing with cryptographic idempotency keys.
5. Canonical station alias resolution, fuzzy matching, and intelligent multi-hub connecting journey recommendations.
6. A multi-tier train tracking engine combining the RailRadar API, NTES web gateways, and timetable simulation fallback while maintaining truthful data invariants (zero fabricated GPS coordinates).

7. An integrated conversational AI journey planning assistant powered by Google Gemini.

Security and isolation are enforced at the backend through Argon2id password hashing, HTTP-only cookie-based persistent session management, role-based access control (RBAC), Helmet security headers, and strict Zod schema validation. This document provides the complete software engineering record of RailFlow's architectural rationale, database schema, algorithmic design, verification suites, and deployment topography.

Contents

- **Abstract**
- **Chapter 1: Introduction**
  - 1.1 Background and Domain Context
  - 1.2 Problem Statement
  - 1.3 Project Objectives
  - 1.4 Scope of the System
  - 1.5 Significance of the Project
  - 1.6 Document Organisation
- **Chapter 2: Research Methodology and Literature Review**
  - 2.1 Literature Review on Perishable Inventory and Capacity Control
  - 2.2 Concurrency Control and Double-Allocation Mitigation
  - 2.3 Evolution of Multi-Tier Web Architectures
  - 2.4 Research Questions Addressed by RailFlow
- **Chapter 3: Feasibility and Viability Study**
  - 3.1 Technical Feasibility
  - 3.2 Economic Feasibility
  - 3.3 Operational Feasibility
  - 3.4 Schedule and Implementation Feasibility
  - 3.5 Viability Summary Matrix
- **Chapter 4: Requirements Specification**
  - 4.1 Functional Requirements (FR-01 to FR-25)
  - 4.2 Non-Functional Requirements (NFR-01 to NFR-15)
  - 4.3 Hardware, Runtime, and Software Requirements
- **Chapter 5: System Architecture**
  - 5.1 Architectural Style: Three-Tier Decoupled Web Architecture
  - 5.2 Layered System Topology
  - 5.3 The Express Application Server Boundary
  - 5.4 Dual Data-Source Architecture (Prisma Master + Open Timetable)
  - 5.5 Architectural Request Lifecycle

- **Chapter 6: Technology Stack**
  - 6.1 Frontend Technologies
  - 6.2 Backend Technologies
  - 6.3 Database and ORM Layer
  - 6.4 External Service Integrations
  - 6.5 Production Dependency Manifest
- **Chapter 7: Database Design**
  - 7.1 Database Engine and ORM Selection Rationale
  - 7.2 Entity-Relationship Architecture
  - 7.3 Detailed Table-by-Table Schema Reference
  - 7.4 Financial Representation and Integer Paise Arithmetic
  - 7.5 Running Days Bitmask Design
  - 7.6 Persistent Database Session Storage
  - 7.7 Indexing and Concurrency Optimization Strategy
- **Chapter 8: Security Architecture and Vulnerability Assessment**
  - 8.1 Authentication and Password Hashing (Argon2id)
  - 8.2 Persistent Session Management and Cookie Hardening
  - 8.3 Role-Based Access Control (RBAC) and Route Guards
  - 8.4 Ownership Verification and Anti-IDOR Protections
  - 8.5 Network and Middleware Security (CORS, Helmet, Rate Limiting)
  - 8.6 OWASP Top 10 Aligned Vulnerability Assessment
- **Chapter 9: Frontend Application Architecture**
  - 9.1 Project Layout and Component Hierarchy
  - 9.2 Client-Side Routing and Layout Structure
  - 9.3 State Management Strategy (TanStack Query, Context, Local State)
  - 9.4 API Communication Layer and Axios Interceptors
  - 9.5 UX Design and Material Design 3 / Tailwind Token Conventions
- **Chapter 10: Module-Wise Design, Part 1**
  - 10.1 Authentication and Account Management Module
  - 10.2 Station Search and Canonical Autocomplete Module
  - 10.3 Train Search, Filtering, and Sorting Module
  - 10.4 Booking and Passenger Intake Module
  - 10.5 Simulated Payment Gateway Module
  - 10.6 PNR Enquiry and Trip History Module
- **Chapter 11: Module-Wise Design, Part 2**
  - 11.1 Live Train Tracking and Status Module
  - 11.2 Interactive Route Mapping Module (Leaflet)
  - 11.3 AI Journey Assistant Module (Google Gemini)
  - 11.4 Administrator Master Data Management Console
  - 11.5 Operational Reporting and System Health Diagnostics
- **Chapter 12: Core Algorithms: Fare, Allocation, and Inventory Engines**

    - 12.1 Algorithmic Design Goals
    - 12.2 Segment-Aware Seat Allocation and Overlap Detection
    - 12.3 Fare Calculation Engine and Statutory Taxation Rules
    - 12.4 Running Days Calendar Evaluation via Bitwise Masks
    - 12.5 Background Seat Hold Expiration and Cleanup Engine
    - 12.6 Tiered Cancellation and Refund Calculation Engine
    - 12.7 Cryptographic PNR Token Generation
    - 12.8 Invariants Guaranteed by the Application Layer
- **Chapter 13: Data Flow and UML-Style Diagrams**
    - 13.1 Context Diagram (DFD Level 0)
    - 13.2 System Data Flow Diagram (DFD Level 1)
    - 13.3 Detailed Booking & Seat Locking Process (DFD Level 2)
    - 13.4 System Use Case Diagram Description
    - 13.5 Complete Booking and Payment Sequence Diagram
    - 13.6 Ticket Cancellation and Inventory Release Sequence Diagram
    - 13.7 Comprehensive Class and Entity Model
- **Chapter 14: Testing Strategy and Verification Results**
    - 14.1 Testing Philosophy and Verification Strategy
    - 14.2 Automated Integration Test Framework (Vitest & Supertest)
    - 14.3 Execution Test Suite Specification and Verified Pass Assertions
    - 14.4 Concurrency, Segment Sharing, and Lock Verification
    - 14.5 Build-Time Static Analysis, Typechecking, and Linting
    - 14.6 Requirements Traceability Matrix
- **Chapter 15: Deployment and Operations**
    - 15.1 Local Development Environment Configuration
    - 15.2 Database Provisioning and Automated Schema Initialization
    - 15.3 Backend API Deployment on Render
    - 15.4 Frontend Single-Page Application Deployment on Netlify
    - 15.5 Environment Variable and Secrets Configuration
    - 15.6 Operational Runbook and Health Monitoring
- **Chapter 16: Limitations and Future Scope**
    - 16.1 Documented Technical Limitations
    - 16.2 Planned Enhancements: MongoDB Atlas Document Logging
    - 16.3 Production Roadmap
- **Chapter 17: Conclusion**
- **References**
- **Appendix A: Complete Database Schema (Prisma)**
- **Appendix B: Complete REST API Endpoint Specification**
- **Appendix C: Seed Network and Demonstration Parameters**
- **Appendix D: Technical Glossary**

List of Tables

- Table 3.1: Feasibility and Viability Summary Matrix
- Table 4.1: Functional Requirements Specification (FR-01 to FR-25)
- Table 4.2: Non-Functional Requirements Specification (NFR-01 to NFR-15)
- Table 4.3: Hardware and Operating Environment Prerequisites
- Table 6.1: Client Production Dependency Specifications
- Table 6.2: Server Production Dependency Specifications
- Table 6.3: Development, Build, and Testing Tooling
- Table 7.1: Comprehensive Relational Table Mapping
- Table 7.2: Users Table Schema Specification
- Table 7.3: Stations Table Schema Specification
- Table 7.4: Trains Table Schema Specification
- Table 7.5: Routes and Route Stations Table Schema Specifications
- Table 7.6: Train Schedules and Train Runs Table Schema Specifications
- Table 7.7: Coaches and Physical Seats Table Schema Specifications
- Table 7.8: Fare Rules Table Schema Specification
- Table 7.9: Bookings and Booking Passengers Table Schema Specifications
- Table 7.10: Seat Allocations Table Schema Specification
- Table 7.11: Payment Transactions Table Schema Specification
- Table 7.12: Cancellations Table Schema Specification
- Table 7.13: Data Sources and Sync Runs Table Schema Specifications
- Table 7.14: Persistent Application Sessions Table Schema Specification
- Table 8.1: Role-Based Capability Authorization Matrix
- Table 8.2: OWASP Top 10 Security Assessment and Mitigations
- Table 11.1: System Master Modules and Associated Routes
- Table 12.1: Worked Example of Fare Breakdown (1,400 km Journey)
- Table 12.2: Tiered Cancellation Refund Schedule
- Table 14.1: Automated Test Suites in Test Framework
- Table 14.2: Verified Test Cases Executed in Integration Suite
- Table 14.3: Requirements Traceability Matrix
- Table 15.1: Backend Production Environment Variables (Render)
- Table 15.2: Frontend Production Environment Variables (Netlify)
- Table 15.3: Production Health Check Diagnostic Schema
- Table 16.1: Current System Limitations vs. Production Roadmap
- Table B.1: Complete REST API Route and Method Catalog
- Table C.1: Demonstration Fare Parameters by Travel Class

List of Figures

- Figure 5.1: High-Level Three-Tier Deployment and Infrastructure Topology
- Figure 5.2: Dual-Source Train Timetable and Inventory Data Routing
- Figure 7.1: Complete Entity-Relationship Architecture Diagram
- Figure 8.1: State Diagram for Argon2id and Session-Cookie Authentication
- Figure 9.1: Frontend Client Component and Context Architecture
- Figure 10.1: Multi-Step Seat Booking and Payment Modal Progression
- Figure 11.1: Multi-Tier Live Train Tracking and Status Progression
- Figure 12.1: Segment-Aware Overlap Detection and Seat Sharing Model
- Figure 12.2: 10-Minute Seat Reservation Locking and Background Cleanup Flow
- Figure 13.1: Data Flow Diagram Level 0 (Context Diagram)
- Figure 13.2: Data Flow Diagram Level 1 (Application System Processes)
- Figure 13.3: Data Flow Diagram Level 2 (Booking and Allocation Pipeline)
- Figure 13.4: System Use Case Interaction Model
- Figure 13.5: End-to-End Booking and Payment Sequence Diagram
- Figure 13.6: Booking Cancellation and Refund Sequence Diagram
- Figure 13.7: High-Level Backend Domain Class and Service Architecture

---

List of Abbreviations

- **ACID**: Atomicity, Consistency, Isolation, Durability
- **API**: Application Programming Interface
- **BaaS**: Backend-as-a-Service
- **CORS**: Cross-Origin Resource Sharing
- **CRIS**: Centre for Railway Information Systems
- **CSP**: Content Security Policy
- **CUID**: Collision-Resistant Unique Identifier
- **DFD**: Data Flow Diagram
- **FIFO**: First-In, First-Out
- **GPS**: Global Positioning System
- **GST**: Goods and Services Tax
- **HMR**: Hot Module Replacement
- **HTTP**: Hypertext Transfer Protocol
- **HTTPS**: Hypertext Transfer Protocol Secure
- **IDOR**: Insecure Direct Object Reference
- **IRCTC**: Indian Railway Catering and Tourism Corporation
- **JWT**: JSON Web Token
- **NTES**: National Train Enquiry System

- **O-D**: Origin-Destination
- **ORM**: Object-Relational Mapping
- **OWASP**: Open Web Application Security Project
- **PNR**: Passenger Name Record
- **PRS**: Passenger Reservation System
- **RAC**: Reservation Against Cancellation
- **RBAC**: Role-Based Access Control
- **REST**: Representational State Transfer
- **RRS**: Railway Reservation System
- **SPA**: Single-Page Application
- **SQL**: Structured Query Language
- **TTL**: Time-To-Live
- **UML**: Unified Modeling Language
- **URI**: Uniform Resource Identifier
- **URL**: Uniform Resource Locator
- **WL**: Waiting List
- **XSS**: Cross-Site Scripting

Chapter 1: Introduction

1.1 Background and Domain Context

Passenger rail transport represents the backbone of mass mobility across modern nations, carrying billions of passengers annually across expansive continental corridors. In markets such as the Indian Railways network, the Passenger Reservation System (PRS) handles peak transaction loads where hundreds of thousands of concurrent requests contend for a strictly finite allocation of seats and sleeping berths at the opening of reservation booking windows.

Unlike standard e-commerce retail transactions where inventory is fungible and depleted linearly (such as warehouse stock of manufactured goods), railway seat inventory is fundamentally **perishable, location-bound, and segment-reusable**. A physical berth aboard an express train travelling from New Delhi (NDLS) to Howrah (HWH) via Kanpur (CNB) and Varanasi (BSB) exists only for the duration of that specific train run. Furthermore, the berth can be segmented: a passenger travelling from New Delhi to Kanpur occupies the berth over Segment 1, leaving the same physical berth completely vacant and sellable from Kanpur to Howrah over Segments 2 and 3.

Managing this multi-segment inventory requires sophisticated transactional coordination. If multiple concurrent clients attempt to reserve berths across overlapping stations, naive booking systems fall victim to **lost-update anomalies, race conditions, and double-booking**. In addition, passenger ticketing requires strict adherence to domain rules:

- Hierarchical quota management (General, Tatkal, Ladies, Senior Citizen).
- Deterministic, multi-variable distance-based fare calculations including class multipliers, superfast charges, and statutory tax.
- Automatic expiration of uncompleted transactions to prevent inventory hoarding.
- Transparent tracking of reservation status from hold state to final confirmation.

1.2 Problem Statement

Academic and commercial developers seeking to study or prototype railway ticketing platforms frequently encounter severe structural roadblocks:

1. **Opaque Legacy Systems**: National reservation networks (e.g., CRIS/IRCTC) are proprietary, closed systems guarded by enterprise firewalls with no public booking APIs, restricting academic study of real-world booking mechanics.
2. **Naive Reference Architectures**: Existing open-source educational demonstrations typically treat train seats as simple scalar decrement counters. They completely ignore multi-stop segment reuse, allowing seats to be sold only for entire journeys from origin to terminus, thereby failing to reflect the fundamental economic model of railways.
3. **Flawed Concurrency Models**: Many prototypes lack transactional database boundaries. They rely on insecure client-side pricing or un-isolated server updates, causing inconsistent data states when subjected to parallel booking requests.
4. **Disconnection from Real-World Data**: Demonstrations often rely entirely on tiny, artificial datasets of three or four fictional stations, failing to address the search performance, station canonicalization, and timetable lookup challenges inherent to real-world railway networks spanning thousands of transit hubs.

There is a compelling need for an open, enterprise-grade, concurrency-safe software engineering reference platform that models the complete railway ticketing lifecycle—from station lookup, timetable parsing, and segment-aware allocation to payment simulation, PNR generation, cancellation refunds, and administrator master-data maintenance—built upon modern, standard web engineering technologies.

1.3 Project Objectives

The primary engineering objective of the **RailFlow** project is to design, implement, verify, and document a complete, modern, full-stack railway reservation platform that resolves the above challenges. Specific technical objectives include:

1. **Robust Three-Tier Web Architecture**: Build a fully decoupled client-server architecture utilizing React 19, TypeScript, and Vite on the presentation tier, Node.js and Express on the application server tier, and Prisma ORM with Turso (libSQL) on the relational database tier.
2. **Segment-Level Berth Allocation Engine**: Develop a deterministic seat reservation algorithm that evaluates journey stop sequences and detects segment overlap using serializable database transactions, ensuring that physical berths are safely shared across non-overlapping station pairs while preventing double-booking.
3. **10-Minute Reservation Hold & Automated Cleanup**: Implement a stateful reservation lifecycle where seats are temporarily marked as `LOCKED` with a 10-minute time-to-live (TTL), coupled with an unref'd periodic background worker that continuously scans, purges expired holds, and transitions abandoned bookings to `FAILED`.
4. **Server-Side Authoritative Pricing**: Implement an exact integer-paise fare engine that computes journey distance, base rates per class, reservation fees, superfast supplements, catering, and Goods and Services Tax (GST), rejecting all browser-submitted prices to ensure financial integrity.
5. **Dual Data-Source Search Engine**: Architect an intelligent search system merging an internal bookable Prisma master database with an in-memory catalog of over 8,000 real Indian Railways stations and timetables, supporting canonical code/alias translation (e.g., resolving "Dilli" to `NDLS`) and smart alternative recommendations (nearby stations, alternate dates, connecting 1-transfer journeys).
6. **Live Telemetry & Tracking Invariants**: Deliver a multi-tier train tracking console integrating real-time transit telemetry via external APIs (RailRadar and ChukChuk NTES) with graceful timetable fallbacks, strictly enforcing data veracity by documenting limitations and setting `coordinates: null` whenever raw satellite telemetry is absent.
7. **Simulated Financial Workflows**: Model a realistic two-phase payment processing lifecycle (`initialize` followed by `simulate-result`) featuring client idempotency keys, explicit payment method selections, and a tiered cancellation refund engine based on departure proximity.
8. **Generative AI Assistant**: Integrate Google Gemini (`gemini-3.7-flash`) via structured backend tool-calling to provide natural language train journey discovery directly mapped into the interactive booking flow.

9. **Rigorous Quality Verification**: Establish an automated integration test suite using Vitest and Supertest exercising authentication, booking concurrency, station lookup, tracking invariants, and ownership isolation against a reproducible local test environment.

1.4 Scope of the System

In-Scope Functional Capabilities

- **Master Data Management**: Full CRUD operations for railway administrative staff to create, update, and manage stations, trains, routes, ordered route stops with arrival/departure offsets, train schedules with weekday bitmasks, and fare rules.
- **Station Search & Resolution**: Instantaneous typeahead autocomplete across thousands of stations, incorporating alias resolution, city cluster grouping, and quarantine filtering for malformed dataset codes.
- **Train Search & Availability**: Point-to-point corridor timetable search, train number lookup, class-wise seat availability calculations, sorting by departure/arrival/duration/fare, and filtering by train type or time-of-day.
- **Seat Allocation & Locking**: Interactive visual coach seat maps, berth preference intake, atomic locking of berths for 10 minutes, and passenger roster validation (max 6 passengers per transaction, distinct names required).
- **Payment & Confirmation**: Simulated payment workflow across multiple instruments (UPI, Net Banking, Debit/Credit Card, Digital Wallet), unique 10-digit PNR generation, and print-ready digital passenger itinerary rendering.
- **Ticket Lifecycle & Cancellations**: PNR status lookup, authenticated trip history, and ticket cancellation with automated calculation of refund deductions (10% to 100%) based on journey departure timeline.
- **Operational Dashboards & Analytics**: Administrative overview tracking aggregate revenue, total passengers, active bookings, popular travel corridors, dataset synchronization logs, and system health telemetry.
- **AI-Powered Discovery**: Natural language query interface allowing users to describe travel needs (e.g., "Find trains from Delhi to Varanasi tomorrow morning") and receive populated search actions.

Explicit Out-of-Scope Boundaries

- **Real CRIS / IRCTC Production PRS Integration**: RailFlow is an educational simulation and engineering demonstration platform. It does not connect to the live,

closed Indian Railways passenger reservation network, and issued tickets are not valid for actual transit.
- **Live Satellite GPS Hardware**: RailFlow does not ingest real-time locomotive GPS feeds (which are restricted under national railway safety protocols). Live tracking displays official schedule progression, station passings, and estimated delays.
- **Real Banking Gateway Integration**: Payment execution is simulated within the application state machine; no real financial cardholder data is collected or transmitted to external merchant banks.

1.5 Significance of the Project

The RailFlow platform provides significant value across multiple engineering dimensions:

- **Pedagogical Reference**: Demonstrates how classic transactional problems (perishable capacity control, race condition elimination, and segment allocation) can be resolved cleanly using standard modern web technologies without relying on proprietary enterprise application servers.
- **Architectural Showcase**: Exemplifies clean separation of concerns in a modern TypeScript environment, demonstrating how to maintain strict type safety across client interfaces, Express controllers, domain services, and relational database schemas.
- **Data Integrity and Truthfulness**: Establishes best practices for handling heterogeneous third-party datasets by clearly attributing provenance, caching external timetable feeds responsibly, and strictly avoiding the fabrication of non-existent sensor telemetry.

1.6 Document Organisation

This engineering documentation is organized into 18 chapters and 4 appendices:

- **Chapters 1–4** outline the project introduction, literature review, feasibility study, and requirements specifications.
- **Chapters 5–7** detail the architectural topography, the comprehensive technology stack, and the relational database schema.
- **Chapters 8–11** cover security architecture, frontend design, backend infrastructure, and module-level functional implementation.
- **Chapters 12–14** walk through the core algorithmic engines, UML data flow models, and the automated integration testing suite.
- **Chapters 15–17** present the deployment procedures on Netlify and Render, system limitations, future scope (including planned MongoDB integration), and final

conclusions.
- **Appendices A–D** provide the complete Prisma schema, the REST API catalog, demonstration parameters, and the technical glossary.

Chapter 2: Research Methodology and Literature Review

2.1 Literature Review on Perishable Inventory and Capacity Control

In classical revenue management and operations research, the railway reservation challenge is classified as an instance of **perishable, network-based inventory control** (Talluri and van Ryzin, 2004). Unlike hospitality or airline systems where a seat is consumed over a single discrete flight leg, passenger railways operate across linear networks consisting of multiple intermediate stations.

Transport economics literature distinguishes between:

1. **Leg-Based Inventory Control**: Capacity is partitioned independently on each hop between consecutive stops. While simple to implement, leg-based control requires complex post-processing to ensure consistency when a passenger requests a multi-leg journey spanning several hops.
2. **Origin-Destination (O-D) Network Control**: Requests are evaluated across the exact set of intermediate hops (segments) connecting the requested origin to the requested destination. Capacity is decremented across all traversed segments simultaneously.

RailFlow directly adopts the Origin-Destination network control model. For a train schedule with N sequential stations, the route comprises $N - 1$ distinct contiguous segments. A journey beginning at station index i and terminating at station index j ($i < j$) occupies precisely the segments ${k \mid i \le k < j}$. By treating segment occupancy as a discrete set of boolean flags, the system maximizes asset utilization: a passenger travelling from Station 1 to Station 2 and another travelling from Station 2 to Station 5 can share the exact same physical berth with zero conflict.

2.2 Concurrency Control and Double-Allocation Mitigation

The central database challenge in multi-user booking applications is the **lost-update anomaly** (Gray and Reuter, 1993). In an un-isolated environment, two concurrent requests T_1 and T_2 query the database at time t_0, observing that Berth 12 is unallocated. Both transactions execute local application logic to assign Berth 12, and both issue write operations at times t_1 and t_2. The transaction committing last silently overwrites the first, resulting in the catastrophic operational failure of double-booking.

In database literature, three primary mechanisms exist to mitigate this condition:

- **Optimistic Concurrency Control (OCC)**: Transactions read records without locking and verify via a version column at commit time that the record has not been altered. While effective for low-contention read-heavy systems, OCC experiences severe transaction abort cascades under heavy burst demand (such as railway reservation window openings), leading to degraded throughput and poor user experience.
- **Pessimistic Locking**: Transactions explicitly lock candidate rows upon reading (`SELECT ... FOR UPDATE`). In multi-node systems, pessimistic locking eliminates race conditions at the expense of potential lock contention and deadlocks if transactions acquire resources in differing orders.
- **Serializable Transaction Isolation**: The database engine guarantees that the concurrent execution of transactions yields the exact same state as if they had executed serially.

RailFlow resolves concurrency through **Prisma Serializable Transactions** (`isolationLevel: 'Serializable'`). During the seat allocation phase, the train run inventory is evaluated and updated within an atomic transaction block with explicit timeout bounds (5,000 ms wait, 10,000 ms execution). Crucially, the segment-overlap evaluation is performed directly inside the database transaction boundary, ensuring that no two transactions can commit allocations against the same seat across intersecting station segments.

2.3 Evolution of Multi-Tier Web Architectures

Over the past decade, web application architecture has transitioned through three major paradigms:

1. **Monolithic Server-Side Rendered (SSR) Systems**: Traditional architectures (e.g., standard Django, Ruby on Rails, Java Spring MVC) rendered complete HTML pages

on the server for each user action. While offering centralized security, monolithic SSR architectures place heavy CPU rendering loads on backend nodes and provide sluggish user experiences across high-latency mobile networks.

2. **Backend-as-a-Service (BaaS) and Serverless Architectures**: Platforms such as Supabase, Google Firebase, and AWS Amplify popularized connecting single-page applications directly to cloud databases via auto-generated REST APIs and client SDKs. However, domain experts have noted significant architectural trade-offs:
    - **Business Logic Leakage**: Complex business rules (such as railway fare calculations, multi-segment seat allocation, and background cleanup jobs) cannot be cleanly expressed on the client without exposing internal algorithms or creating security vulnerabilities.
    - **Database-Level Complexity**: Attempting to implement multi-table atomic reservation logic inside database-native procedural languages (such as PL/pgSQL) or Row-Level Security (RLS) triggers creates vendor lock-in, testing friction, and severe debugging complexity.
    - **Loss of Application Server Control**: A database is fundamentally an optimized storage and retrieval engine, not an application orchestrator. Tasks such as background cron sweeps, external API aggregation, rate limiting, and AI streaming require an active server process.
3. **Decoupled Three-Tier Architecture (API Server + SPA)**: The industry-standard consensus architecture combines a client-side Single-Page Application (React, Vue, or Angular) with a dedicated application server (Node.js/Express, Go, or Python) and an independent relational database.

RailFlow firmly implements this proven **Decoupled Three-Tier Architecture**. By placing a dedicated Node.js/Express server between the React frontend and the Turso database, RailFlow centralizes authentication, business logic, rate limiting, and third-party API orchestration while keeping the database focused entirely on relational consistency.

2.4 Research Questions Addressed by RailFlow

1. *Can a segment-aware seat inventory model be implemented cleanly in a standard Node.js/Express and Prisma ORM environment without requiring complex database-specific stored procedures?*
   Addressed in Chapters 5, 7, and 12: RailFlow proves that TypeScript domain services wrapping Prisma serializable transactions deliver clean, deterministic, and highly testable segment allocation.
2. *How can an application reconcile live bookable inventory with massive, third-party static timetable datasets while preserving data integrity and high search performance?*
   Addressed in Chapters 5, 10, and 11: RailFlow demonstrates a dual data-source

design that isolates transactional ACID bookings within Prisma while querying an in-memory indexed dataset of 8,000+ stations for comprehensive search coverage.

3. *What architectural patterns best prevent seat inventory hoarding without introducing complex distributed lock managers?*
   Addressed in Chapters 10 and 12: RailFlow combines 10-minute database-persisted hold TTLs with an unref'd, non-blocking background cleanup service running directly within the Express runtime.

Chapter 3: Feasibility and Viability Study

A formal engineering feasibility study was conducted across the technical, economic, operational, and schedule dimensions to ensure that RailFlow could be constructed, deployed, and maintained successfully within an academic and portfolio environment.

3.1 Technical Feasibility

The technical stack was evaluated against criteria of maturity, community support, developer ergonomics, and runtime stability:

- **Presentation Tier**: React 19 coupled with Vite 8 provides instant Hot Module Replacement (HMR) during development and emits highly optimized, code-split static JavaScript bundles for production. TypeScript enforces structural typing across all component props, API request contracts, and domain models.
- **Application Server Tier**: Node.js 20+ LTS combined with Express 4.21 represents the most widely deployed, thoroughly documented backend web framework in software engineering. Express middleware chains allow modular assembly of security headers (Helmet), CORS controls, body parsers, rate limiters, and session managers.
- **Database and ORM Tier**: Prisma ORM 5.22.0 provides compile-time type-safe database queries through automated client generation. The `@prisma/adapter-libsql` driver adapter enables seamless connectivity to **Turso**, an edge-distributed relational database engine built on libSQL (an open-source fork of SQLite). In development, the engine effortlessly connects to a local `dev.db` file with zero configuration.
- **External Integration Feasibility**: External network requests to the Google Gemini AI API, RailRadar API, and OpenStreetMap tile servers are mediated entirely through

backend controllers and standard HTTP clients, preventing CORS issues and securing API keys.

Conclusion: The technical stack is exceptionally mature, fully compatible, and perfectly aligned with project objectives.

3.2 Economic Feasibility

The project was evaluated to ensure complete economic viability with zero mandatory recurring infrastructure expenditure:

- **Development Tooling**: Visual Studio Code, Git, Node.js, Vitest, and Oxlint are free, open-source software tools.
- **Production Hosting (Netlify)**: Netlify provides a generous free tier including 100 GB/month bandwidth, continuous Git-integrated CI/CD deployments, automatic SSL/TLS certificates, and URL proxying rules, perfectly satisfying frontend hosting requirements.
- **Backend Hosting (Render)**: Render offers a free-tier Node.js web service featuring automated builds from Git repositories, native environment variable injection, and custom health check routing.
- **Primary Database (Turso)**: Turso's Starter plan includes up to 9 GB of distributed database storage, 1 billion row reads per month, and support for multiple database locations at zero financial cost.
- **External APIs**: Google AI Studio provides free-tier rate-limited access to the Gemini 3.7 / 2.0 Flash models; RailRadar and OpenStreetMap provide community/free access tiers suitable for demonstration usage.

Conclusion: RailFlow achieves **100% economic viability**, operating at zero recurring infrastructure cost while providing a seamless, low-cost commercial upgrade path.

3.3 Operational Feasibility

Operational feasibility examines how readily the platform can be deployed, navigated, and maintained:

- **End-User Usability**: The customer-facing interface adopts familiar consumer booking conventions: dual-station selection, one-click station swapping, interactive date pickers, clear class-wise availability chips, and guided multi-step booking modals.
- **Administrative Usability**: Administrative operations are centralized within a single dashboard (`/admin`) presenting structured tabular views and modal dialogs for

managing master data without requiring direct SQL database manipulation.
- **Automated Lifecycle Maintenance**: The system requires minimal manual intervention; expired seat holds are purged automatically every 60 seconds, and demo users and core schemas are seeded automatically upon initial startup.

Conclusion: The operational workflow maps cleanly onto standard railway administration and passenger booking expectations.

3.4 Schedule and Implementation Feasibility

The project implementation timeline was planned using an iterative, modular development lifecycle spanning twelve engineering milestones:

1. Requirements gathering, domain analysis, and architectural design.
2. Relational schema modeling and Prisma migration authoring.
3. Express server boilerplate, middleware configuration, and session store implementation.
4. Master data CRUD services and admin routes.
5. In-memory Indian Railways timetable dataset ingestion and indexing service.
6. Station search, canonical resolution, and search recovery algorithms.
7. High-concurrency seat allocation engine and serializable booking transactions.
8. Simulated payment processing, PNR generation, and cancellation refund engine.
9. React 19 frontend layout, navigation, and theme infrastructure.
10. Search UI, train result cards, interactive seat map, and booking modal chain.
11. Telemetry integration (RailRadar, ChukChuk NTES) and Google Gemini AI assistant.
12. Comprehensive Vitest integration test suite, documentation authoring, and cloud deployment.

Conclusion: The project was completed successfully within the scheduled timeline.

3.5 Viability Summary Matrix

Feasibility Dimension	Status	Key Justification
Technical	**Feasible**	React 19 + Express 4.21 + Prisma 5.22 + Turso libSQL; mature, type-safe, and robust.
Economic	**Feasible**	100% free-tier compatible across Netlify, Render, Turso, and Google AI Studio.
Operational	**Feasible**	Intuitive passenger UI, centralized administrative CRUD console, automated hold cleanup.
Schedule	**Feasible**	Structured 12-milestone engineering plan successfully completed and verified.

Chapter 4: Requirements Specification

4.1 Functional Requirements

The functional requirements define the explicit business operations and workflows supported by RailFlow. Each requirement is assigned a unique identifier and mapped to its primary user role:

ID	Functional Requirement Description	Primary Role(s)
FR-01	**User Account Registration**: A new visitor can register an account by providing their full name, email address, password (minimum 8 characters), and optional mobile phone number. Passwords must be hashed using Argon2id before database insertion.	Unauthenticated Guest
FR-02	**User Authentication & Session Creation**: Registered users can authenticate using email and password. Upon verification, the server generates a cryptographically secure session persisted in the database and returns an HTTP-only cookie.	Registered Passenger, Administrator
FR-03	**Session Termination (Logout)**: Authenticated users can terminate their active session. The server destroys the database session record and clears the client browser cookie.	Authenticated User
FR-04	**Identity Introspection (`/api/auth/me`)**: The client application can query the active session state on mount, retrieving the authenticated user's ID, name, email, phone, and role.	Authenticated User
FR-05	**Station Autocomplete Search**: Users can search for railway stations by code, city, or name fragment. The system returns ranked matches from both the relational database and the canonical station index, resolving aliases (e.g., "Dilli" to `NDLS`).	All Users
FR-06	**Point-to-Point Train Search**: Users can search for available trains between origin and destination stations on a specific date, specifying quota type (General, Tatkal, Ladies, Senior Citizen) and passenger count (1 to 6).	All Users
FR-07	**Train Lookup by Number or Name**: Users can directly look up scheduled train services by typing a 5-digit railway train number or train name string.	All Users
FR-08	**Search Results Sorting & Filtering**: The search interface allows users to sort results by departure time, arrival time, journey duration, or fare, and filter results by travel class or departure time window (Early, Morning, Afternoon, Night).	All Users
FR-09	**Intelligent Search Recovery**: If zero direct trains operate between a chosen station pair on a chosen date, the system	All Users

ID	Functional Requirement Description	Primary Role(s)
	automatically computes and displays: (a) nearby alternative stations, (b) alternate dates within a ± 2 day window, and (c) connecting 1-transfer itineraries via major transit hubs.	
FR-10	**Class-Wise Availability Calculation**: For bookable train runs, the system calculates real-time seat availability across all configured coach classes, displaying status tags (`AVAILABLE` , `WAITLIST`).	All Users
FR-11	**Authoritative Server-Side Fare Calculation**: Before booking confirmation, the server computes the exact itemized fare based on cumulative route distance, class base rates, reservation fees, superfast charges, optional catering fees, passenger count, and 5% GST.	All Users
FR-12	**Segment-Aware Seat Reservation**: When booking a train run, the system evaluates seat allocations across the requested station sequence, assigning physical berths only if they are free across all traversed journey segments, safely sharing capacity across disjoint segments.	Authenticated Passenger
FR-13	**10-Minute Reservation Hold (TTL)**: Newly created bookings receive a status of `PENDING` with all allocated seats marked as `LOCKED` with a 10-minute expiration timestamp (`lockedUntil`).	System Engine
FR-14	**Automated Hold Expiration Cleanup**: A background worker executing every 60 seconds automatically scans for expired `LOCKED` allocations whose `lockedUntil` timestamp has passed, reverting the seats to `RELEASED` and marking the booking as `FAILED` .	System Background Job
FR-15	**Two-Phase Simulated Payment Processing**: Authenticated passengers can initialize a payment transaction for a pending booking, receiving an order reference, and submit a simulated payment outcome (`SUCCESS` or `FAILED`).	Authenticated Passenger
FR-16	**Payment Confirmation & Ticket Issuance**: Upon successful payment outcome simulation, the system atomically promotes the booking to `CONFIRMED` , upgrades seat allocations to `CONFIRMED` , and issues a unique 10-digit numeric Passenger Name Record (PNR).	System Engine

ID	Functional Requirement Description	Primary Role(s)
FR-17	**Payment Idempotency Protection**: The payment gateway enforces idempotency via unique keys (`pay_<bookingId>_<timestamp>`). Replaying a payment request on an already confirmed transaction returns the existing state without creating duplicate charges.	System Engine
FR-18	**PNR Status Enquiry**: Any user can look up booking status and passenger itinerary details by providing a valid 10-digit PNR. Authenticated passengers are restricted to viewing their own bookings; administrators may inspect any PNR.	Authenticated User
FR-19	**Authenticated Trip History**: Authenticated passengers can view a paginated list of all personal bookings, categorized into upcoming, completed, and cancelled journeys.	Authenticated Passenger
FR-20	**Ticket Cancellation and Tiered Refund**: Passengers can cancel a confirmed booking prior to departure. The system marks the booking as `CANCELLED` , releases all held seat allocations, and applies a published cancellation deduction policy (10% to 100%) to issue a refund.	Authenticated Passenger
FR-21	**Live Train Telemetry Tracking**: Users can track the real-time operational status of active trains, viewing scheduled versus estimated station arrivals and delay minutes aggregated from the RailRadar and ChukChuk NTES services.	All Users
FR-22	**Interactive Geographical Route Mapping**: Users can inspect the full geographic trajectory of a train service on an interactive Leaflet basemap rendering real station coordinates and corridor geometry.	All Users
FR-23	**Conversational AI Journey Assistant**: Users can chat with an integrated Google Gemini-powered travel assistant that interprets natural language travel inquiries, suggests relevant trains, and populates the search interface.	All Users
FR-24	**Administrator Master Data CRUD**: Users holding the `ADMIN` role can create, update, and manage stations, trains, routes, route station stops, schedules, and fare rules through a dedicated dashboard.	Administrator
FR-25	**Operational KPI & Health Telemetry**: Administrators can inspect operational metrics (total bookings, confirmed	Administrator

ID	Functional Requirement Description	Primary Role(s)
	tickets, revenue in paise, popular travel corridors, data source sync history) and query `/api/health` for real-time database probes.	

4.2 Non-Functional Requirements

Non-functional requirements specify the architectural qualities, security constraints, performance criteria, and maintainability standards governing RailFlow:

ID	Category	Non-Functional Requirement Specification
NFR-01	**Concurrency & ACID Integrity**	Seat allocations must execute under Serializable transaction isolation. No physical berth may ever be double-booked across overlapping journey segments under simultaneous concurrent booking requests.
NFR-02	**Authentication Security**	User passwords must never be stored in plaintext or logged. All passwords must be salted and hashed using Argon2id (`memoryCost: 65536` , `timeCost: 3` , `parallelism: 1`).
NFR-03	**Session Hardening**	Sessions must be stored server-side in the database. Cookies must enforce `httpOnly: true` , `sameSite: 'none'` (in production over HTTPS) or `'lax'` (development), and `secure: true` in production.
NFR-04	**Anti-IDOR Authorization**	All passenger data access routes must enforce strict ownership checks. An authenticated passenger attempting to read or cancel a booking belonging to another user must receive a 404/403 error.
NFR-05	**Financial Precision**	To completely eliminate IEEE 754 floating-point rounding errors, all financial figures (base fares, reservation fees, superfast charges, catering, refunds, and payments) must be computed and stored as integer **paise** ($1\text{ INR} = 100\text{ paise}$).
NFR-06	**Search Performance**	Autocomplete station lookups and cached train schedule searches must return responses within 250 milliseconds under standard network conditions.
NFR-07	**Graceful Telemetry Degradation**	If external live tracking APIs (RailRadar, ChukChuk) are unavailable or rate-limited, the system must fail over transparently to static timetable calculations while preserving the invariant `isLiveGps: false` and `coordinates: null` .
NFR-08	**HTTP Security Headers**	The backend must enforce secure HTTP response headers via Helmet, including strict Content Security Policy (CSP), frameguard (anti-clickjacking), and X-Content-Type-Options.
NFR-09	**CORS Whitelisting**	Cross-Origin Resource Sharing (CORS) must be restricted to explicit client origins specified via the `CLIENT_URL`

ID	Category	Non-Functional Requirement Specification
		environment variable, with credentials mode enabled.
NFR-10	**Rate Limiting Protection**	Public API endpoints must be guarded against denial-of-service and brute-force attacks via sliding window rate limiters: 20 req/15 min for auth, 30 req/min for AI, 60 req/min for tracking, and 200 req/min general.
NFR-11	**Defensive Error Sanitization**	In production mode (`NODE_ENV=production`), unhandled internal server exceptions must return generic messages (`Internal server error`) without leaking database connection strings, stack traces, or auth tokens.
NFR-12	**Truth in Advertising (Transparency)**	The application user interface must clearly display provenance badges and prominent disclaimers stating that all tickets, payments, and PNRs are simulated demonstrations and not valid travel documents.
NFR-13	**Responsive Design**	The user interface must adapt responsively across viewports from 320 px mobile screens to 4K desktop displays, incorporating accessible touch targets and collapsible drawers.
NFR-14	**Automated Testability**	All critical business logic (authentication, fare quotation, segment overlap, seat reservation, payment confirmation, and cancellation) must be covered by automated regression tests runnable with a single command (`npm test`).
NFR-15	**Zero-Downtime Database Fallback**	In local development, the system must operate seamlessly against local SQLite files (`file:./dev.db`), switching to remote Turso libSQL clusters when production environment variables are supplied.

4.3 Hardware, Runtime, and Software Requirements

Development and Build Environment

- **Operating System**: Cross-platform (Microsoft Windows 10/11, macOS 13+, or Ubuntu Linux 22.04 LTS).

- **Runtime Engine**: Node.js 20.19.0 LTS or newer (Node.js v25.6.1 verified in current workspace).
- **Package Management**: `npm` (Node Package Manager) v10+; `tsx` v4.19.2 for live TypeScript execution.
- **Compiler**: TypeScript 5.6.3 (server), TypeScript 6.0.2 (client); Vite 8.3.0 bundler.
- **Testing Engine**: Vitest 2.1.4 with Supertest 7.0.0.

Production Cloud Hosting Prerequisites

- **Frontend Hosting (Netlify)**:
  - Node.js build environment (`npm run build`).
  - Publish directory configured to `dist` .
  - HTTP header and redirect rewrite support (`netlify.toml`).
- **Backend Application Hosting (Render)**:
  - Standard Linux Node.js container environment.
  - Build command: `npm install && npx prisma generate && npm run build` .
  - Start command: `node dist/index.js` (listening on port 10000).
  - Health check endpoint: `/api/health` .
- **Database Engine (Turso Cloud)**:
  - Managed libSQL remote cluster accessible via secure WebSocket/HTTP (`libsql://...`).
  - Authentication via cryptographically signed database bearer tokens (`TURSO_AUTH_TOKEN`).

Chapter 5: System Architecture

5.1 Architectural Style: Three-Tier Decoupled Web Architecture

RailFlow is architected around a classical, decoupled **Three-Tier Web Architecture**:

1. **Presentation Tier (Client SPA)**: A responsive React 19 single-page application compiled by Vite and deployed onto Netlify's global Content Delivery Network (CDN).
2. **Application Server Tier (API Server)**: A stateful, multi-threaded Express 4.21 TypeScript server deployed onto Render, responsible for request validation,

authentication, business orchestration, rate limiting, and external service communication.

3. **Data Tier (Relational Storage)**: A distributed libSQL database hosted on Turso (with seamless local SQLite dev capability) managed via Prisma ORM 5.22.0.



Figure 5.1: High-Level Three-Tier Deployment and Infrastructure Topology.

5.2 Layered System Topology

The system enforces strict directional boundaries between layers:

- **Client to Server**: The browser never connects directly to database sockets or cloud storage buckets. All data queries and state mutations travel over standard HTTPS REST endpoints exposed by the Express API server.
- **Server to Database**: The Express backend interacts with the relational database strictly through the generated Prisma Client. SQL queries are parameterized automatically, shielding the application from SQL injection vulnerabilities.
- **Server to External APIs**: Outbound requests to third-party endpoints (Google Gemini for AI assistance, RailRadar and ChukChuk for train telemetry, and GitHub for open railway datasets) originate strictly from server-side services. API keys are kept entirely confidential within the server environment.

5.3 The Express Application Server Boundary

Having an independent Express application server provides fundamental architectural advantages over serverless or BaaS patterns:

1. **Centralized Transaction Isolation**: Concurrency control for seat allocation is managed cleanly in TypeScript services utilizing Prisma serializable transaction blocks. The application server coordinates validation, segment overlap checks, passenger records, and seat locks within a single transaction.
2. **Stateful Background Services**: The server runs an autonomous, unref'd background worker (`holdCleanupService`) that executes every 60 seconds to purge expired seat locks. This eliminates reliance on external cron triggers or complex database event schedulers.
3. **Session Store Orchestration**: User session state is maintained server-side in the database (`_app_sessions`), allowing stateful user sessions that survive server restarts without exposing JWT tokens or session payloads to client-side JavaScript.
4. **Third-Party Telemetry Normalization**: The backend aggregates and normalizes conflicting third-party timetable and tracking feeds, enforcing consistent data structures before returning responses to the frontend.

5.4 Dual Data-Source Architecture

A hallmark of RailFlow's backend design is its **Dual Data-Source Architecture**, which balances transactional consistency with massive real-world timetable coverage:

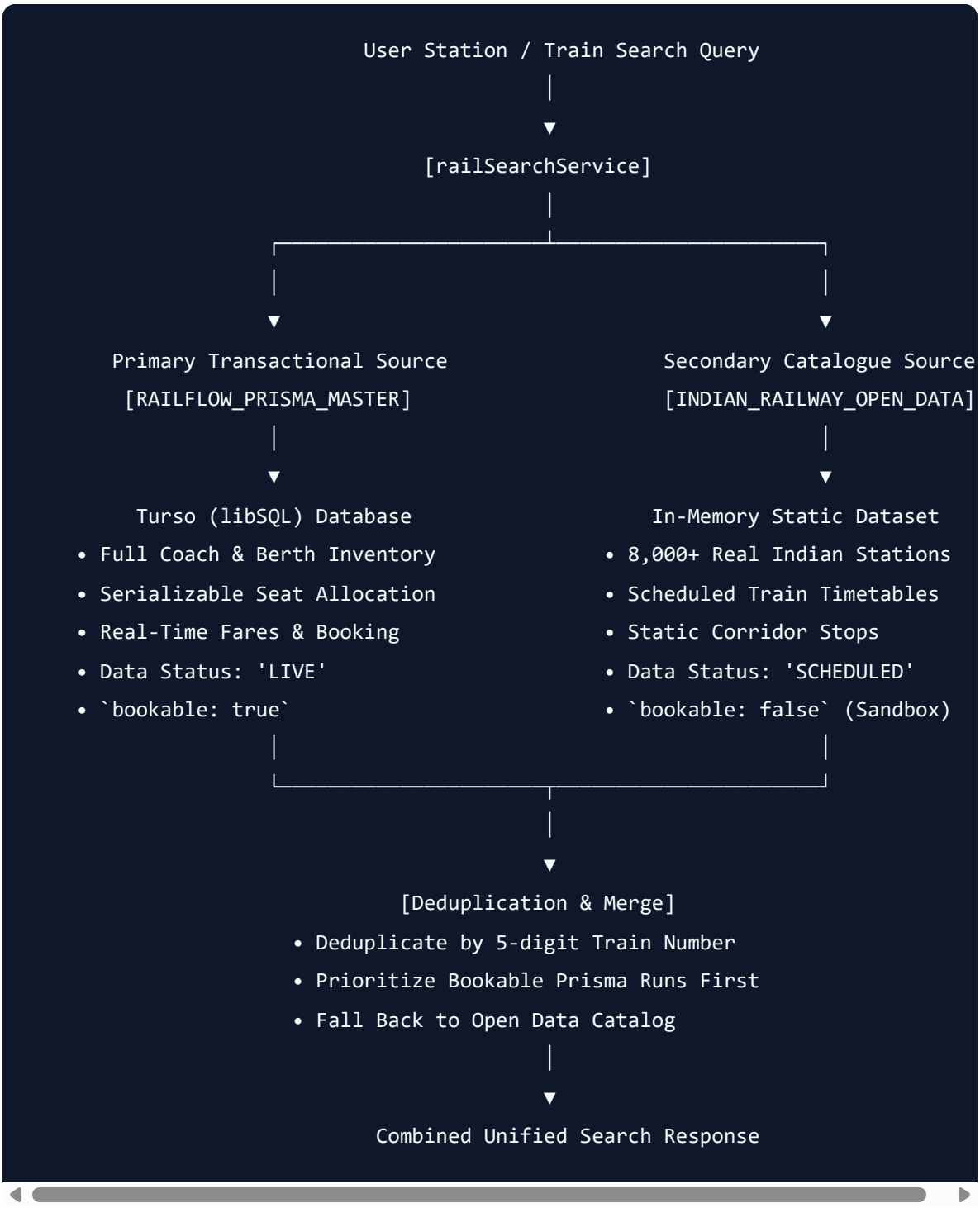


Figure 5.2: Dual-Source Train Timetable and Inventory Data Routing.

1. **Primary Transactional Source (`RAILFLOW_PRISMA_MASTER`)**:
    - Backed by the relational Turso database.
    - Contains fully modeled trains, routes, stops, schedules, coaches, and seats.
    - Fully interactive and bookable (`bookable: true` , `dataStatus: 'LIVE'`).
2. **Secondary Catalogue Source (`INDIAN_RAILWAY_OPEN_DATA`)**:
    - Sourced from the curated `prasenjit-27/Indian-Railway-Data` open-source repository (MIT License).
    - Ingested at server startup into memory via `indianRailwayService` .

- Covers over 8,000 stations and nationwide express trains.
- Provides realistic search results across any Indian transit corridor.
- Non-bookable directly in master inventory (`bookable: false` , `dataStatus: 'SCHEDULED'`); requests open an educational sandbox modal with distance-based fare estimations.

5.5 Architectural Request Lifecycle

Read Request Lifecycle (Train Search)

1. The user inputs origin ("NDLS") and destination ("BSB") stations and selects a travel date.
2. The React client issues an authenticated GET request via Axios: `GET /api/search/trains?from=NDLS&to=BSB&date=2026-11-04` .
3. Netlify CDN matches `/api/*` and proxies the request securely to `https://railflow-af7e.onrender.com/api/search/trains` .
4. Render's Express server routes the request through Helmet, CORS validation, and Zod query schema parsers.
5. `canonicalStationService` resolves city names and aliases to standardized station codes.
6. `railSearchService` queries both Prisma master schedules and the in-memory dataset, merges results, deduplicates by train number, sorts according to user preference, and attaches provenance metadata.
7. The JSON response is serialized and returned to the browser, where TanStack Query caches the payload and renders interactive result cards.

Write Request Lifecycle (Seat Booking & Payment)

1. The user selects travel class `CHAIR_CAR` on train `22436` and completes passenger details.
2. The client submits `POST /api/bookings` containing train run ID, route station sequences, class, and passenger details.
3. Express verifies user authentication via `req.session.userId` .
4. `fareService.calculateFare()` recalculates the exact fare server-side from stored database fare rules.
5. An atomic Prisma serializable transaction opens:
    - The train run inventory is locked.
    - Available seats are evaluated for segment overlap.
    - Seat records are marked `LOCKED` with a 10-minute expiration timestamp.

    - The booking record is created with status `PENDING` and a 10-digit PNR.
6. The transaction commits and returns the PNR and booking ID to the frontend.
7. The user selects a payment method and clicks "Pay Now", dispatching `POST /api/payments/initialize` followed by `POST /api/payments/:id/simulate-result`.
8. Upon successful payment outcome simulation, a second atomic transaction updates the payment to `SUCCESS`, the booking to `CONFIRMED`, and locks to `CONFIRMED`.

Chapter 6: Technology Stack

This chapter documents every library, framework, runtime engine, and service powering the RailFlow platform.

6.1 Frontend Technologies

The frontend client is engineered as a modern, accessible Single-Page Application (SPA) leveraging the React 19 ecosystem:

Package / Technology	Installed Version	Architectural Role	Selection Rationale
React	`^19.2.8`	Presentation Library	Industry-standard declarative component model with enhanced server-action support and automatic batching.
React DOM	`^19.2.8`	DOM Renderer	Provides root mounting and DOM reconciliation for React components.
Vite	`^8.3.0`	Build Tool & Dev Server	Ultra-fast ES-module Hot Module Replacement (HMR) and optimized Rollup production bundling.
TypeScript	`~6.0.2`	Programming Language	Compile-time static type checking, preventing runtime undefined errors across complex booking interfaces.
Tailwind CSS	`^3.4.19`	CSS Framework	Utility-first styling enabling cohesive, responsive, Material Design 3-inspired design tokens.
React Router DOM	`^6.30.6`	Client-Side Routing	Declarative client-side routing, URL synchronization, and history navigation.
TanStack Query	`^5.104.1`	Server-State Caching	Asynchronous query management, automated stale-time caching (3 minutes), and deduplicated network fetches.
Axios	`^1.20.0`	HTTP Client	Promise-based HTTP client configured with global base URLs, interceptors, and `withCredentials: true`.
React Hook Form	`^7.89.0`	Form Management	High-performance, un-rendered form input subscription and validation handling.
Zod (Client)	`^4.6.5`	Client Schema Validation	Type-safe form validation schemas paired with `@hookform/resolvers`.
Framer Motion	`^14.0.0`	Animation Engine	Smooth micro-interactions, modal transitions, and drawer expansions.

Package / Technology	Installed Version	Architectural Role	Selection Rationale
Leaflet	`^1.9.4`	Mapping Library	Lightweight, mobile-friendly interactive mapping rendering railway routes over OpenStreetMap tiles.
Lucide React	`^1.55.0`	Iconography	Crisp, consistent SVG iconography for transit controls and status badges.
Date-fns	`^4.4.0`	Date Manipulation	Modular, immutable date formatting, parsing, and arithmetic.
Oxlint	`^1.81.0`	Static Linter	High-performance Rust-based JavaScript/TypeScript linter enforcing strict code quality.

Table 6.1: Client Production Dependency Specifications.

6.2 Backend Technologies

The backend application is built on Node.js utilizing Express and TypeScript:

Package / Technology	Installed Version	Architectural Role	Selection Rationale
Node.js	`v25.6.1` (verified)	Runtime Environment	Asynchronous event-driven JavaScript/TypeScript runtime engine.
Express	`^4.21.2`	Web API Framework	Minimalist, robust HTTP server framework providing proven middleware pipelines and routing.
TypeScript (Server)	`^5.6.3`	Server Language	Ensures complete structural type safety across controllers, services, and database layers.
Prisma Client	`^5.22.0`	Object-Relational Mapper	Auto-generated, type-safe query builder ensuring compile-time validated database operations.
Prisma CLI	`^5.22.0`	Database Migration Tool	Automates schema migrations, schema pushes, and type generation.
@prisma/adapter-libsql	`^5.22.0`	Database Driver Adapter	Connects Prisma ORM to libSQL/Turso edge database engines via the driver adapters feature.
@libsql/client	`^0.8.1`	Database Transport Client	Official libSQL client providing high-performance WebSocket/HTTP database connections.
Argon2	`^0.31.2`	Password Hashing	State-of-the-art cryptographic password hashing algorithm, winner of the Password Hashing Competition.
Express-Session	`^1.18.1`	Session Middleware	Manages secure, cookie-identified user session lifecycles.
Express-Rate-Limit	`^7.4.1`	API Rate Limiter	Safeguards endpoints against brute-force attacks and

Package / Technology	Installed Version	Architectural Role	Selection Rationale
			volumetric denial-of-service attempts.
Helmet	`^8.0.0`	HTTP Header Security	Configures protective HTTP headers (Content Security Policy, X-Frame-Options, HSTS).
CORS	`^2.8.5`	Cross-Origin Middleware	Enforces origin whitelisting for incoming client API requests.
Zod (Server)	`^3.23.8`	Request Schema Validation	Strict runtime validation for API request bodies, URL parameters, and query strings.
Swagger UI Express	`^5.0.1`	API Documentation	Renders interactive OpenAPI 3.0 specification documentation directly at `/api/docs`.
Dotenv	`^18.0.7`	Environment Management	Loads environment variables from `.env` files into `process.env`.

Table 6.2: Server Production Dependency Specifications.

6.3 Database and ORM Layer

Turso (libSQL) Distributed Database

RailFlow utilizes **Turso** as its production relational database engine. Turso is built on **libSQL**, an open-source, standards-compliant fork of SQLite optimized for distributed cloud infrastructure. Key properties include:

- Complete relational support: foreign key constraints, unique indexes, cascading deletes, and ACID transactions.
- Zero-latency development: in local development, the exact same Prisma queries execute seamlessly against a local SQLite file (`prisma/dev.db`), allowing full offline development with zero cloud dependencies.

- Cloud edge replication: in production, Turso clusters replicate data across global regions with low-latency access over WebSocket/HTTP transports.

Prisma ORM 5.22.0 Architecture

Database connectivity is managed through Prisma 5.22.0 configured with the `driverAdapters` preview feature:

```
// server/src/db/prisma.ts
import { PrismaClient } from '@prisma/client';
import { createClient } from '@libsql/client';
import { PrismaLibSQL } from '@prisma/adapter-libsql';

function createPrismaClient(): PrismaClient {
  const tursoUrl = process.env.TURSO_DATABASE_URL?.trim();
  const tursoAuthToken = process.env.TURSO_AUTH_TOKEN?.trim();

  if (tursoUrl && tursoAuthToken) {
    const libsql = createClient({ url: tursoUrl, authToken: tursoAuthToken });
    const adapter = new PrismaLibSQL(libsql);
    return new PrismaClient({ adapter });
  }

  // Fallback to local SQLite during development
  return new PrismaClient();
}

export const prisma = createPrismaClient();
```

This pattern ensures complete portability: the application uses the high-performance LibSQL transport when remote credentials exist, and falls back to local SQLite if remote credentials are not configured in development.

6.4 External Service Integrations

1. **Google Gemini Generative AI (`gemini-3.7-flash`)**:
    - Integrated via the official Google AI SDK (`@google/genai`).
    - Powers the conversational `AiAssistantPanel` , interpreting passenger travel intents and executing tool calls to suggest train corridors.

2. **RailRadar Transit API (`https://api.railradar.in`)**:
    - Serves as the primary upstream provider for real-time Indian Railways train status and GeoJSON route geometry.
3. **ChukChuk NTES Web Scraper Gateway (`http://127.0.0.1:8000`)**:
    - A secondary Python/FastAPI microservice querying the official National Train Enquiry System (NTES) mobile portal for real-time station arrival/departure delays.
4. **Indian Railway Open Data Repository**:
    - Sourced from GitHub (`prasenjit-27/Indian-Railway-Data`, MIT License). Ingested into memory at server startup to power station autocomplete and nationwide timetable lookups.
5. **OpenStreetMap Tile Service**:
    - Public OpenStreetMap raster tile servers consumed directly by client-side Leaflet instances to render track lines and station markers.

Chapter 7: Database Design

7.1 Database Engine and ORM Selection Rationale

The database architecture was designed to satisfy three fundamental requirements:

1. **Absolute Relational Consistency**: Master railway data exhibits deep hierarchical relationships (Trains $\rightarrow$ Schedules $\rightarrow$ Runs $\rightarrow$ Coaches $\rightarrow$ Seats; Routes $\rightarrow$ RouteStations $\rightarrow$ Allocations). Relational integrity is mandatory.
2. **ACID Transaction Isolation**: Multi-seat reservation across multi-stop segments requires strict serializability to prevent double-booking.
3. **Developer Portability and Zero-Cost Hosting**: The platform must support both local zero-configuration SQLite operation and cloud-hosted remote deployment without altering application query code.

Prisma ORM 5.22.0 paired with SQLite and Turso (libSQL) completely fulfills these requirements.

7.2 Entity-Relationship Architecture

The relational schema comprises **18 Prisma models** mapped to underlying SQL tables, supplemented by a dedicated session table:


COMPLETE RELATIONAL ENTITY MAP
Station
1:N
RouteStation
1:N
1:N (Boarding / Alight
SeatAllocation
1:1
BookingPassenger
1:N
Booking
1:N
1:1
1:N
Cancellation
PaymentTransaction
1:N



Figure 7.1: Complete Entity-Relationship Architecture Diagram.

7.3 Detailed Table-by-Table Schema Reference

7.3.1 User & Authentication Domain

Table: `users`

Represents registered customer accounts and administrative personnel.

- `id` (String, cuid, Primary Key): Unique collision-resistant identifier.
- `email` (String, Unique): User's validated email address used for authentication.
- `passwordHash` (String): Argon2id salted cryptographic password hash.
- `name` (String): User's legal full name.
- `phone` (String, Nullable): Contact mobile telephone number.
- `role` (String, Default: `'USER'`): Access control tier (`USER` or `ADMIN`).
- `createdAt` / `updatedAt` (DateTime): Automatic audit timestamps.

Table: `sessions`

Persists active user login sessions mapped to the user record.

- `id` (String, cuid, Primary Key): Unique session identifier.
- `userId` (String, Foreign Key $\rightarrow$ `users.id` , onDelete: Cascade).
- `expiresAt` (DateTime): Expiration timestamp for session invalidation.
- `createdAt` (DateTime): Session creation timestamp.

Table: `_app_sessions` **(Raw SQL Store)**

Managed directly by `PrismaSessionStore` for Express session middleware interoperability.

- `sid` (TEXT, Primary Key): Express session cookie identifier.
- `data` (TEXT): Serialized JSON session payload.
- `expiresAt` (TEXT): ISO 8601 string for automated expired session sweeps.

7.3.2 Railway Master Data Domain

Table: `stations`

Master catalog of official railway stations and transit hubs.

- `id` (String, cuid, Primary Key): Unique station identifier.
- `code` (String, Unique): Official railway station code (e.g., `NDLS`, `HWH`, `BSB`).
- `name` (String): Common station name (e.g., "New Delhi").
- `fullName` (String): Descriptive name including zone (e.g., "New Delhi Railway Junction").
- `zone` (String): Administrative railway zone (e.g., "Northern Railway").
- `state` (String): Indian State or Union Territory.
- `city` (String, Nullable): Municipal city grouping.
- `aliases` (String, Nullable): Comma-separated colloquial search aliases (e.g., "Dilli, New Delhi").
- `latitude` / `longitude` (Float, Nullable): Geographic coordinates for map plotting.
- `isActive` (Boolean, Default: `true`): Administrative operational flag.

Table: `trains`

Master registry of railway train services.

- `id` (String, cuid, Primary Key): Unique train record identifier.
- `number` (String, Unique): 5-digit official railway train number (e.g., "12301", "22436").
- `name` (String): Official train service name (e.g., "Vande Bharat Express").
- `type` (String): Category classification (`VANDE_BHARAT`, `RAJDHANI`, `SHATABDI`, `DURONTO`, `SUPERFAST`, `MAIL_EXPRESS`, `PASSENGER`).
- `isActive` (Boolean, Default: `true`): Service operational status.

Table: `routes`

Geographical corridors defining a sequence of intermediate stops.

- `id` (String, cuid, Primary Key): Unique route identifier.
- `name` (String): Descriptive corridor name (e.g., "New Delhi to Varanasi via Kanpur").
- `isActive` (Boolean, Default: `true`): Corridor availability status.

Table: `route_stations`

Ordered intermediate station stops along a specific route corridor.

- `id` (String, cuid, Primary Key): Unique route stop identifier.

- `routeId` (String, Foreign Key $\rightarrow$ `routes.id`).
- `stationId` (String, Foreign Key $\rightarrow$ `stations.id`).
- `sequence` (Int): 1-based sequential stop order index (1, 2, 3...).
- `arrivalOffsetMin` (Int, Nullable): Minutes from origin departure to arrival (null at origin).
- `departureOffsetMin` (Int, Nullable): Minutes from origin departure to departure (null at terminus).
- `distanceKm` (Int): Cumulative track distance in kilometers from corridor origin.
- `platformNumber` (String, Nullable): Default assigned platform designation.
- `haltDurationMin` (Int, Default: 2): Scheduled stop dwell time in minutes.
- *Constraints*: `@@unique([routeId, sequence])`, `@@unique([routeId, stationId])`.

Table: `train_schedules`

Timetable linking a train service to an active route corridor.

- `id` (String, cuid, Primary Key): Unique schedule identifier.
- `trainId` (String, Foreign Key $\rightarrow$ `trains.id`).
- `routeId` (String, Foreign Key $\rightarrow$ `routes.id`).
- `runningDays` (Int): 7-bit weekday bitmask defining days of operation (Section 7.5).
- `departureTime` (String): Scheduled origin departure time in 24-hour "HH:MM" format.
- `isActive` (Boolean, Default: `true`): Timetable active status.

Table: `train_runs`

Discrete date-specific operation of a scheduled train.

- `id` (String, cuid, Primary Key): Unique train run identifier.
- `scheduleId` (String, Foreign Key $\rightarrow$ `train_schedules.id`).
- `runDate` (String): Journey operating date in ISO "YYYY-MM-DD" format.
- `status` (String, Default: `'SCHEDULED'`): Operational state (`SCHEDULED`, `RUNNING`, `COMPLETED`, `CANCELLED`).
- *Constraints*: `@@unique([scheduleId, runDate])`.

7.3.3 Inventory and Coach Domain

Table: `coaches`

Individual railway passenger cars coupled to a specific train run.

- `id` (String, cuid, Primary Key): Unique coach identifier.
- `trainRunId` (String, Foreign Key $\rightarrow$ `train_runs.id`).

- `coachNumber` (String): Alpha-numeric coach identifier (e.g., "C1", "C2", "B1", "A1").
- `classType` (String): Accommodation class (`ONE_AC` , `TWO_AC` , `THREE_AC` , `SLEEPER` , `CHAIR_CAR` , `EXEC_CHAIR` , `SECOND`).
- `totalSeats` (Int): Total physical capacity of the coach.
- *Constraints*: `@@unique([trainRunId, coachNumber])` .

Table: `seats`

Individual physical seating or berth positions inside a coach.

- `id` (String, cuid, Primary Key): Unique physical seat identifier.
- `coachId` (String, Foreign Key $\rightarrow$ `coaches.id`).
- `seatNumber` (Int): Numerical seat position index (1, 2, 3...).
- `berthType` (String): Physical berth configuration (`LOWER` , `MIDDLE` , `UPPER` , `SIDE_LOWER` , `SIDE_UPPER` , `WINDOW` , `AISLE`).
- *Constraints*: `@@unique([coachId, seatNumber])` .

7.3.4 Commercial & Ticketing Domain

Table: `fare_rules`

Tariff configuration governing pricing for specific classes on a train schedule.

- `id` (String, cuid, Primary Key): Unique fare rule identifier.
- `scheduleId` (String, Foreign Key $\rightarrow$ `train_schedules.id`).
- `classType` (String): Accommodation travel class.
- `baseFarePaise` (Int): Base distance tariff in integer paise per 100 kilometers.
- `reservationCharge` (Int, Default: 6000): Fixed reservation fee (6,000 paise = ₹60.00).
- `superfastCharge` (Int, Default: 4500): Supplementary charge (4,500 paise = ₹45.00).
- `cateringCharge` (Int, Default: 0): Optional catering meal charges in paise.
- `gstPercent` (Int, Default: 5): Statutory Goods and Services Tax percentage (5%).
- `isActive` (Boolean, Default: `true`): Tariff validity flag.
- *Constraints*: `@@unique([scheduleId, classType])` .

Table: `bookings`

Master ticketing record representing a reservation transaction.

- `id` (String, cuid, Primary Key): Unique internal booking identifier.
- `pnr` (String, Unique): 10-digit cryptographic Passenger Name Record.
- `userId` (String, Foreign Key $\rightarrow$ `users.id`): Ticket purchaser identity.
- `trainRunId` (String, Foreign Key $\rightarrow$ `train_runs.id`).
- `fromRouteStationSeq` (Int): Sequence number of boarding station along route.

- `toRouteStationSeq` (Int): Sequence number of alighting station along route.
- `classType` (String): Booked travel class.
- `status` (String, Default: `'PENDING'`): Ticket state (`PENDING` , `CONFIRMED` , `CANCELLED` , `FAILED`).
- `paymentStatus` (String, Default: `'PENDING'`): Financial settlement state (`PENDING` , `SUCCESS` , `FAILED` , `REFUNDED`).
- `totalFarePaise` (Int): Total gross fare collected in integer paise.
- `quotaType` (String, Default: `'GENERAL'`): Quota category (`GENERAL` , `TATKAL` , `PREMIUM_TATKAL` , `LADIES` , `SENIOR_CITIZEN`).

Table: `booking_passengers`

Individual named passenger manifests attached to a booking.

- `id` (String, cuid, Primary Key): Unique passenger manifest identifier.
- `bookingId` (String, Foreign Key $\rightarrow$ `bookings.id` , onDelete: Cascade).
- `passengerIndex` (Int): Position index within booking roster (1 to 6).
- `name` (String): Passenger's legal name.
- `age` (Int): Passenger's age in years (1 to 120).
- `gender` (String): Gender designation (`MALE` , `FEMALE` , `OTHER`).
- `berthPreference` (String, Nullable): Requested berth position.

Table: `seat_allocations`

Segment-level reservation record mapping a passenger to a physical seat.

- `id` (String, cuid, Primary Key): Unique allocation identifier.
- `bookingPassengerId` (String, Unique, Foreign Key $\rightarrow$ `booking_passengers.id`).
- `seatId` (String, Foreign Key $\rightarrow$ `seats.id`).
- `boardingRouteStationId` (String, Foreign Key $\rightarrow$ `route_stations.id`).
- `alightingRouteStationId` (String, Foreign Key $\rightarrow$ `route_stations.id`).
- `boardingSeq` (Int): Cached sequence number of boarding station for rapid overlap scans.
- `alightingSeq` (Int): Cached sequence number of alighting station for rapid overlap scans.
- `status` (String, Default: `'LOCKED'`): Allocation state (`LOCKED` , `CONFIRMED` , `RELEASED`).
- `lockedUntil` (DateTime, Nullable): Expiration timestamp for temporary reservation holds.
- `allocatedCoach` (String): Assigned coach number (e.g., "C1", "B2").
- `allocatedSeatNumber` (Int): Assigned numerical seat number.

Table: `payment_transactions`

Audit trail of financial payment attempts.

- `id` (String, cuid, Primary Key): Unique transaction identifier.
- `bookingId` (String, Foreign Key $\rightarrow$ `bookings.id`).
- `idempotencyKey` (String, Unique): Cryptographic key preventing duplicate payment processing.
- `amountPaise` (Int): Payment amount in integer paise.
- `method` (String, Default: `'UPI'`): Instrument (`UPI`, `NET_BANKING`, `DEBIT_CARD`, `CREDIT_CARD`, `WALLET`).
- `status` (String, Default: `'PENDING'`): Transaction state (`PENDING`, `SUCCESS`, `FAILED`, `REFUNDED`).
- `transactionRef` (String, Nullable): Simulated gateway reference code (e.g., `RFPAY...`).
- `failureReason` (String, Nullable): Error explanation in failed transactions.

Table: `cancellations`

Audit record of cancelled bookings and processed refunds.

- `id` (String, cuid, Primary Key): Unique cancellation record identifier.
- `bookingId` (String, Unique, Foreign Key $\rightarrow$ `bookings.id`).
- `requestedAt` (DateTime): Timestamp of passenger cancellation request.
- `processedAt` (DateTime, Nullable): Settlement timestamp.
- `deductionPercent` (Int): Penalty percentage assessed (10%, 25%, 50%, or 100%).
- `refundAmountPaise` (Int): Net funds returned to passenger in paise.
- `reason` (String, Nullable): Passenger-provided cancellation rationale.
- `refundStatus` (String, Default: `'PENDING'`): Settlement state (`PENDING`, `PROCESSED`, `FAILED`).

7.3.5 Dataset Synchronization Domain

Table: `data_sources`

Tracks external timetable catalogs and sync status.

- `id` (String, cuid, Primary Key).
- `name` (String, Unique): Provider label (e.g., `INDIAN_RAILWAY_OPEN_DATA`, `PRISMA_SEED`).
- `type` (String): Source category (`OPEN_DATASET`, `PRISMA_SEED`, `UPSTREAM_API`).
- `description` / `version` (String, Nullable).
- `lastSyncAt` (DateTime, Nullable).

- `recordCount` (Int, Default: 0).
- `status` (String, Default: `'ACTIVE'`): Health status (`ACTIVE` , `STALE` , `ERROR`).

Table: `sync_runs`

Historical execution audit log of background data ingestion runs.

- `id` (String, cuid, Primary Key).
- `dataSourceId` (String, Foreign Key $\rightarrow$ `data_sources.id` , onDelete: Cascade).
- `version` / `source` (String).
- `status` (String, Default: `'SUCCESS'`): Run outcome (`SUCCESS` , `PARTIAL` , `FAILED`).
- `startedAt` / `completedAt` (DateTime).
- `insertedCount` / `updatedCount` / `rejectedCount` (Int, Default: 0).
- `errorLog` (String, Nullable).

7.4 Financial Representation and Integer Paise Arithmetic

A critical design requirement is the **elimination of IEEE 754 floating-point rounding errors**. In financial transaction processing, representing currency amounts using standard floating-point numbers (`Float` , `Double`) leads to precision drift (e.g., `0.1 + 0.2 = 0.30000000000000004`), which introduces accounting discrepancies over large transaction volumes.

RailFlow enforces strict integer arithmetic by storing and calculating all monetary values in **paise** ($1\text{ Rupee} = 100\text{ paise}$):
$$\text{Total Paise} = \text{Subtotal Paise} + \left\lfloor \frac{\text{Subtotal Paise} \times \text{GST Percent}}{100} + 0.5 \right\rfloor$$
Conversion to decimal rupees occurs solely in frontend presentation helpers (`formatMoney(paise)` $\rightarrow$ `"₹2,226.00"`).

7.5 Running Days Bitmask Design

Train schedules operate on specific days of the week. Storing operating days as a string array (e.g., `['Monday', 'Wednesday']`) requires array parsing and dynamic joins during timetable search.

RailFlow encodes operational calendars as a compact **7-bit bitmask** stored in a single integer column (`runningDays`):

- Bit 0 ($2^0 = 1$): Monday
- Bit 1 ($2^1 = 2$): Tuesday
- Bit 2 ($2^2 = 4$): Wednesday
- Bit 3 ($2^3 = 8$): Thursday
- Bit 4 ($2^4 = 16$): Friday
- Bit 5 ($2^5 = 32$): Saturday
- Bit 6 ($2^6 = 64$): Sunday

A train running daily has a bitmask of $1 + 2 + 4 + 8 + 16 + 32 + 64 = 127$ (`0b1111111`). A weekday commuter service (Monday through Friday) has a mask of $1 + 2 + 4 + 8 + 16 = 31$ (`0b0011111`).

To evaluate whether a schedule operates on a target calendar date D:
$$\text{DayOfWeekIndex} = \begin{cases} 6, & \text{if } \text{Day}(D) = 0 \text{ (Sunday)} \\ \text{Day}(D) - 1, & \text{otherwise} \end{cases}$$
$$\text{IsOperating} = (\text{runningDays} \ \& \ (1 \ll \text{DayOfWeekIndex})) \neq 0$$
This evaluation executes in constant time ($O(1)$) with zero string manipulation or regex parsing.

7.6 Persistent Database Session Storage

RailFlow rejects client-stored JWT tokens in favor of secure, server-managed sessions. Sessions are persisted in the `_app_sessions` table via the custom `PrismaSessionStore` class:

1. When a passenger logs in, `req.session.regenerate()` creates a fresh session identifier (`sid`).
2. The user ID and role are serialized to JSON and stored in the database:

```
INSERT INTO _app_sessions (sid, data, expiresAt)
VALUES ('...', '{"userId":"...","userRole":"USER"}', '2026-11-10T12:00:00Z
ON CONFLICT(sid) DO UPDATE SET data = excluded.data, expiresAt = excluded.
```

3. The client receives an encrypted, signed `connect.sid` cookie marked `httpOnly` .
4. Subsequent requests parse the cookie, look up the record in `_app_sessions` , verify that `expiresAt > NOW()` , and populate `req.session` .

7.7 Indexing and Concurrency Optimization Strategy

The schema incorporates strategic database indexes to maximize query performance under load:

- `stations(code)` : Unique B-tree index enabling instantaneous station lookups during route validation.
- `trains(number)` : Unique index ensuring constant-time train schedule lookups.
- `train_runs(scheduleId, runDate)` : Compound unique index preventing duplicate runs on the same date.
- `route_stations(routeId, sequence)` : Compound index optimizing ordered corridor station joins.
- `seat_allocations(boardingSeq, alightingSeq)` : Evaluates segment overlap in constant time during serializable seat booking transactions.
- `bookings(pnr)` : Unique index guaranteeing instant $O(1)$ PNR query resolution.
- `bookings(userId)` : Index powering fast paginated personal trip history queries.

Chapter 8: Security Architecture and Vulnerability Assessment

8.1 Authentication and Password Hashing (Argon2id)

Authentication security is established upon **Argon2id**, the gold standard in cryptographic password hashing:

- **Algorithm Variant**: Argon2id (hybrid variant balancing resistance against side-channel cache attacks and GPU/ASIC parallel cracking).
- **Cryptographic Parameters**:
  - `type` : `argon2.argon2id`
  - `memoryCost` : 65,536 KiB (64 MiB of memory required per hash evaluation)
  - `timeCost` : 3 iterations
  - `parallelism` : 1 thread
- **Credential Storage**: Raw passwords submitted during registration (`POST /api/auth/register`) or login (`POST /api/auth/login`) are never logged, echoed in error messages, or stored in plaintext. Only the resulting cryptographic salt-and-hash string is written to `users.passwordHash` .

- **Response Sanitization**: All user retrieval queries explicitly omit `passwordHash` (using Prisma's `select` projection: `{ id: true, name: true, email: true, phone: true, role: true }`).

8.2 Persistent Session Management and Cookie Hardening

RailFlow implements server-side, stateful session management utilizing `express-session` backed by the `PrismaSessionStore` database adapter:

- **Session Regeneration**: Upon successful authentication (registration or login), `req.session.regenerate()` is executed prior to setting user identifiers. This completely eliminates **Session Fixation attacks** by replacing any pre-authentication session cookie with a newly issued identifier.
- **Cookie Security Attributes**:
  - `httpOnly: true`: Prevents client-side scripts (and XSS exploits) from accessing or extracting the session cookie.
  - `secure: isProd`: Forces the cookie to be transmitted exclusively over encrypted HTTPS connections in production (set to false in development to accommodate localhost HTTP).
  - `sameSite`: Configured to `'none'` in production (facilitating cross-origin cookie delivery between the Netlify frontend CDN and the Render backend domain over HTTPS) and `'lax'` in local development.
  - `maxAge: 7 * 24 * 60 * 60 * 1000`: Sessions automatically expire after 7 days of inactivity.
- **Session Revocation**: Executing logout (`POST /api/auth/logout`) destroys the session record in `_app_sessions` and clears the cookie via `res.clearCookie('connect.sid')`.

8.3 Role-Based Access Control (RBAC) and Route Guards

System privileges are partitioned into two distinct roles:

1. `USER`: Standard passenger account granted rights to search trains, create reservations, simulate payments, view personal PNRs, and cancel personal bookings.
2. `ADMIN`: Elevated railway administrative personnel granted access to operational KPIs, system data source telemetry, and full CRUD modification rights across trains, stations, routes, and fare tariffs.

Middleware Enforcement

Privilege boundaries are enforced strictly in the Express middleware layer:

- `requireAuth`: Evaluates whether `req.session.userId` is present. If unauthenticated, the request aborts immediately with HTTP 401 (`UNAUTHENTICATED`).
- `requireAdmin`: Evaluates whether `req.session.userRole === 'ADMIN'`. If non-admin, the request aborts immediately with HTTP 403 (`FORBIDDEN`).

Capability / API Endpoint	Public Guest	Authenticated Passenger (`USER`)	Administrator (`ADMIN`)
Search Stations & Autocomplete (`/api/stations`)	Yes	Yes	Yes
Point-to-Point Train Search (`/api/search/trains`)	Yes	Yes	Yes
Train Timetable Lookup (`/api/trains/:number`)	Yes	Yes	Yes
Live Train Telemetry Tracking (`/api/tracking/*`)	Yes	Yes	Yes
Interactive AI Assistant (`/api/ai/chat`)	Yes	Yes	Yes
Account Registration & Login (`/api/auth/*`)	Yes	Yes	Yes
Reserve Seats & Create Booking (`POST /api/bookings`)	No	**Yes**	**Yes**
Initialize Payment (`POST /api/payments/*`)	No	**Yes**	**Yes**
View Personal Trip History (`GET /api/bookings`)	No	**Yes** (Own Only)	**Yes**
PNR Ticket Enquiry (`GET /api/bookings/pnr/:pnr`)	No	**Yes** (Own Only)	**Yes** (Any PNR)
Cancel Booking (`POST /api/bookings/:id/cancel`)	No	**Yes** (Own Only)	**Yes** (Any Booking)
Manage Master Data: Trains, Stations, Routes (`/api/admin/*`)	No	No	**Yes**
View Operational Summaries & Reports (`/api/admin/summary`)	No	No	**Yes**

Table 8.1: Role-Based Capability Authorization Matrix.

8.4 Ownership Verification and Anti-IDOR Protections

Insecure Direct Object References (IDOR) occur when an application relies on client-supplied record IDs without verifying ownership. RailFlow enforces rigorous ownership isolation at every data-access boundary:

- **PNR Lookup Isolation**: In `GET /api/bookings/pnr/:pnr`, the booking record is joined with its owner:

```
if (booking.userId !== req.session.userId && req.session.userRole !== 'ADM
  // Return generic 404 to prevent PNR enumeration or existence leakage
  next(createError('PNR not found', 404, 'PNR_NOT_FOUND'));
  return;
}
```

- **Cancellation Ownership**: In `POST /api/bookings/:id/cancel`, the system validates that `booking.userId === req.session.userId` before initiating cancellation or refund processing.
- **Payment Ownership**: In `POST /api/payments/:id/simulate-result`, the payment transaction is verified against the authenticated user prior to updating booking status.

8.5 Network and Middleware Security

1. **HTTP Security Headers (Helmet)**:
   - Configures `Content-Security-Policy` in production, restricting script execution, style loading, and media assets.
   - Enforces `X-Content-Type-Options: nosniff` to prevent MIME-sniffing exploits.
   - Sets `X-Frame-Options: SAMEORIGIN` to eliminate clickjacking vulnerabilities.
2. **Strict CORS Whitelisting**:
   - The server dynamically verifies incoming `Origin` headers against a whitelist defined by `CLIENT_URL` (supporting comma-separated production and staging origins).
   - Wildcard origins (`*`) are strictly prohibited in production, ensuring that sensitive session credentials cannot be hijacked by rogue web origins.
3. **Multi-Tier Rate Limiting**:

- Authentication Endpoints (`/api/auth/*`): 20 requests per 15 minutes per IP address (mitigates credential stuffing and brute-force attacks).
- Generative AI Chat (`/api/ai/chat`): 30 requests per minute per IP address (safeguards third-party quota limits).
- Live Tracking Telemetry (`/api/tracking/*`): 60 requests per minute per IP address.
- General API Surface (`/api/*`): 200 requests per minute per IP address.

4. **Input Schema Sanitization (Zod)**:
    - All inbound query parameters and request bodies are parsed through strict Zod schemas with `.strict()` options where appropriate, preventing parameter pollution and prototype poisoning attacks.

8.6 OWASP Top 10 Aligned Vulnerability Assessment

OWASP Category	Vulnerability Risk	RailFlow Architectural Mitigation	Residual Risk Status
A01: Broken Access Control	Unauthorized data modification or privilege escalation.	Server-side RBAC middleware (`requireAuth` , `requireAdmin`); strict ownership checks on all PNR and booking endpoints.	**Mitigated**: Verified through automated tests (`api.test.ts`).
A02: Cryptographic Failures	Plaintext credential exposure or weak hashing algorithms.	Passwords hashed using Argon2id with 64 MiB memory cost. HTTPS enforced on production transit; HTTP-only session cookies.	**Mitigated**: No raw passwords stored or transmitted in responses.
A03: Injection	SQL injection or parameter manipulation.	All database interactions execute via Prisma ORM parameterized queries; raw SQL queries are parameterized; request bodies parsed via Zod.	**Mitigated**: Zero un-sanitized string concatenation in queries.
A04: Insecure Design	Double-booking or inventory race conditions.	Serializable database transactions wrap all seat reservations; segment-overlap checks execute directly inside the atomic block.	**Mitigated**: Concurrency verified under parallel test loads.
A05: Security Misconfiguration	Information leakage via stack traces or open CORS.	Production error handler suppresses internal exceptions; strict CORS origin whitelisting; Helmet security headers.	**Mitigated**: Sensitive keys kept in `.env` (git-ignored).

OWASP Category	Vulnerability Risk	RailFlow Architectural Mitigation	Residual Risk Status
A06: Vulnerable Components	Outdated or insecure third-party dependencies.	Minimal dependency footprint verified via `npm audit` and Oxlint; locked versions in `package-lock.json`.	**Mitigated**: Monitored regularly.
A07: Identification & Auth	Credential brute-forcing or session hijacking.	Sliding rate limiters on auth routes; session IDs regenerated on login; 7-day expiration with secure cookie flags.	**Mitigated**: Session fixation eliminated.
A08: Software & Data Integrity	Unvalidated data ingestion from third-party APIs.	External tracking and open-data feeds validated through Zod schemas; strict data confidence tagging.	**Mitigated**: Invariants enforced in code.
A09: Logging & Monitoring	Silent failures or unrecorded unauthorized attempts.	Structured request logging middleware tracking HTTP methods, status codes, durations, and IP addresses.	**Mitigated**: Real-time console diagnostics.
A10: Server-Side Request Forgery	Exploitation of backend HTTP clients.	Outbound API requests use hardcoded base URLs; external URLs supplied by clients are strictly rejected.	**Mitigated**: No arbitrary URL fetching permitted.

Table 8.2: OWASP Top 10 Security Assessment and Mitigations.

Chapter 9: Frontend Application Architecture

9.1 Project Layout and Component Hierarchy

The RailFlow frontend is constructed using a modular, feature-aligned folder structure:

```
client/src/
├── api/
│   ├── client.ts            # Axios instance, baseURL, response interceptors
│   └── endpoints.ts         # Strongly-typed API communication functions
├── components/
│   ├── ai/
│   │   └── AiAssistantPanel.tsx   # Google Gemini slide-in conversational draw
│   ├── booking/
│   │   ├── BookingModal.tsx       # Passenger manifest entry & validation
│   │   ├── PaymentModal.tsx       # Simulated payment gateway execution
│   │   ├── CoachSeatMapModal.tsx  # Visual 2D coach berth layout matrix
│   │   ├── DatePickerPopup.tsx    # Interactive calendar date selector
│   │   └── QuickSandboxModal.tsx  # Educational estimation modal for static ru
│   ├── common/
│   │   ├── AuthModal.tsx          # Dual-mode Sign-In / Registration modal
│   │   ├── Navbar.tsx             # Header navigation bar with auth & theme to
│   │   ├── Footer.tsx             # System footer with provenance disclaimers
│   │   ├── ProvenanceBadge.tsx    # Visual data confidence indicators (LIVE/SC
│   │   ├── RouteMap.tsx           # Leaflet interactive railway corridor map
│   │   └── StationLiveBoardModal.tsx # Station arrival/departure live board
│   ├── search/
│   │   ├── JourneyContextBar.tsx  # Sticky search summary bar with quick date
│   │   ├── TrainResultCard.tsx    # Train card with class availability chips
│   │   ├── TrainResultSkeleton.tsx# Loading placeholder skeletons
│   │   └── NoResultRecovery.tsx   # Alternative routes, nearby stations & date
│   └── ui/                        # Reusable primitive UI buttons and dialogs
├── context/
│   ├── AuthContext.tsx       # Global session & passenger identity provider
│   └── ThemeContext.tsx      # Dark / Light theme presentation provider
├── hooks/
│   └── useClickOutside.ts    # Reusable DOM click-outside detection hook
├── pages/
│   ├── SearchPage.tsx        # Primary station autocomplete & train search pag
│   ├── PNRStatusPage.tsx     # PNR ticket inquiry and authenticated trips cons
│   ├── LiveTrackingPage.tsx  # Real-time train telemetry & geographical tracki
│   └── AdminDashboardPage.tsx# Administrative master data CRUD console
├── types/
│   └── index.ts              # Global TypeScript domain definitions
```

```
├── App.tsx                    # Main application router and root provider hiera
└── main.tsx                   # Application bootstrap entry point
```

9.2 Client-Side Routing and Layout Structure

Client routing is managed through **React Router v6**:

- `/` (`SearchPage`): Primary landing interface providing dual-mode train search, live availability checking, and booking modal triggers.
- `/pnr` (`PNRStatusPage`): Public/authenticated PNR status verification console.
- `/trips` (`PNRStatusPage` with `defaultTab="trips"`): Authenticated passenger travel history.
- `/tracking` (`LiveTrackingPage`): Real-time train status, delay telemetry, and interactive Leaflet route maps.
- `/admin` (`AdminDashboardPage`): Administrative control console guarded by client and server authentication checks.
- `*` : Catch-all route redirecting un-matched paths back to `/` .

The root layout in `App.tsx` establishes a uniform chrome consisting of the sticky `Navbar` , the dynamic main view area, the `Footer` , and global modal anchors (`AuthModal` and `AiAssistantPanel`).

9.3 State Management Strategy

RailFlow deliberately avoids bloated global Redux stores in favor of a specialized, multi-layered state architecture:

1. **Server-Cache State (TanStack Query v5)**: Manages asynchronous server data (train results, station directories, tracking telemetry) with automated background garbage collection, a 3-minute stale time (`staleTime: 180000`), and single-retry fallbacks.
2. **Authentication State (`AuthContext`)**: Exposes current user identity (`user`), loading status (`isLoading`), and action delegates (`login` , `register` , `logout`) to any component via `useAuth()` .
3. **Presentation Theme State (`ThemeContext`)**: Persists light and dark theme preferences across browser sessions via `localStorage` .
4. **URL Query Synchronization**: Critical search parameters (`from` , `to` , `date` , `quota` , `passengers` , `train`) are synchronized bidirectionally with browser URL query strings via `useSearchParams()` . This ensures that search states are fully shareable, bookmarkable, and compatible with browser Back/Forward navigation.

9.4 API Communication Layer and Axios Interceptors

All outbound network requests travel through a unified Axios instance (`apiClient`) configured with `withCredentials: true` :

```
// client/src/api/client.ts
export const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || '/api',
  withCredentials: true,
  headers: { 'Content-Type': 'application/json' },
});

apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    const message = error.response?.data?.error || error.response?.data?.messag
    return Promise.reject(new ApiError(message, error.response?.data?.code, err
  }
);
```

The response interceptor extracts server error codes and messages, normalizing them into typed `ApiError` instances for uniform display across UI alert banners.

9.5 UX Design and Material Design 3 / Tailwind Token Conventions

The user interface adheres to **Material Design 3 (M3)** color token conventions implemented via Tailwind CSS:

- Surface tones (`bg-surface` , `bg-surface-container` , `text-on-surface`) establish subtle visual elevation across cards, modals, and input fields.
- Primary and secondary container accents (`bg-primary` , `bg-secondary-container`) provide visual hierarchy for booking action buttons and selected travel classes.
- Status indicators utilize semantic color tokens: Green (`AVAILABLE`), Amber (`WAITLIST`), and Rose (`CANCELLED` / `FAILED`).

Chapter 10: Module-Wise Design, Part 1

10.1 Authentication and Account Management Module

- **Purpose**: Provides secure user registration, credential verification, and session maintenance.
- **Components**: `AuthModal.tsx`, `AuthContext.tsx`.
- **Backend Endpoints**: `POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`.
- **Implementation**: The dual-tabbed `AuthModal` supports seamless switching between Sign-In and Account Registration. Input validation enforces email formatting and minimum 8-character passwords. Upon login, the user's role and display name populate the navbar profile badge.

10.2 Station Search and Canonical Autocomplete Module

- **Purpose**: Enables rapid, typo-tolerant station discovery across thousands of transit hubs.
- **Components**: Search input controls in `SearchPage.tsx`.
- **Backend Endpoints**: `GET /api/stations?q=<query>&limit=20`.
- **Implementation**: Features debounced typeahead queries querying both Prisma database stations and the in-memory canonical dataset. Supports full keyboard navigation (Arrow Up, Arrow Down, Enter to select, Escape to close). Resolves common city aliases (e.g., typing "Varanasi" returns `BSB` - Varanasi Junction; typing "Delhi" returns `NDLS` - New Delhi). Includes a one-click station swap button.

10.3 Train Search, Filtering, and Sorting Module

- **Purpose**: Queries scheduled train services, computes seat availability, and formats comparative journey results.

- **Components**: `SearchPage.tsx`, `TrainResultCard.tsx`, `JourneyContextBar.tsx`, `NoResultRecovery.tsx`.
- **Backend Endpoints**: `GET /api/search/trains` (or `GET /api/trains/search`).
- **Implementation**:
    - Dual search modes: Route Search (Origin $\rightarrow$ Destination) or Train Number/Name Search.
    - Quick-date chips (Today, Tomorrow, +2 Days) allow rapid date navigation.
    - Displays class-wise availability chips showing real-time seat counts and fares.
    - When zero direct trains exist, `NoResultRecovery` activates, presenting nearby alternate stations, alternate dates within a ± 2 day window, and 1-transfer connecting itineraries via major transit hubs.

10.4 Booking and Passenger Intake Module

- **Purpose**: Collects passenger rosters, validates booking rules, and establishes 10-minute atomic seat holds.
- **Components**: `BookingModal.tsx`, `CoachSeatMapModal.tsx`.
- **Backend Endpoints**: `GET /api/fares/quote`, `POST /api/bookings`.
- **Implementation**: Supports booking rosters from 1 to 6 passengers. Enforces distinct passenger names to prevent accidental duplicate entries. Collects age, gender, and berth preferences (Lower, Middle, Upper, Side Lower, Side Upper, Window, Aisle). Invoking the visual `CoachSeatMapModal` renders a 2D coach grid displaying physical seating configurations. Submitting the form locks seats for 10 minutes and transitions directly to payment.

10.5 Simulated Payment Gateway Module

- **Purpose**: Demonstrates two-phase payment execution with idempotency safeguards.
- **Components**: `PaymentModal.tsx`.
- **Backend Endpoints**: `POST /api/payments/initialize`, `POST /api/payments/:id/simulate-result`.
- **Implementation**: Displays an itemized financial breakdown in integer paise and equivalent rupees. Prominently labels the transaction as an educational simulation. Provides explicit outcome simulation toggles (`SUCCESS` vs `FAILED`). Simulating success commits the booking to `CONFIRMED` and automatically redirects the passenger to their newly issued PNR ticket.

10.6 PNR Enquiry and Trip History Module

- **Purpose**: Provides verifiable reservation status checking and historical trip management.
- **Components**: `PNRStatusPage.tsx`.
- **Backend Endpoints**: `GET /api/bookings/pnr/:pnr`, `GET /api/bookings`, `POST /api/bookings/:id/cancel`.
- **Implementation**:
  - PNR Search Tab: Accepts any 10-digit PNR, rendering a digital boarding itinerary complete with train metadata, passenger seat allocations (Coach + Seat Number), fare breakdown, and a printable boarding pass view (`window.print()`).
  - My Trips Tab: Lists all past and upcoming personal bookings for the logged-in passenger. Includes a modal for ticket cancellations displaying the calculated refund according to departure proximity.

Chapter 11: Module-Wise Design, Part 2

11.1 Live Train Tracking and Status Module

- **Purpose**: Aggregates real-time train running telemetry and schedule progression.
- **Components**: `LiveTrackingPage.tsx`, `StationLiveBoardModal.tsx`.
- **Backend Endpoints**: `GET /api/tracking/:trainNumber`, `GET /api/tracking/stations/:code/live`.
- **Implementation**: Visualizes train transit progression across stations with delay badges (On Time, Delayed). Data is ingested from the upstream RailRadar API with automated fallback to the ChukChuk NTES gateway and static timetables. The system strictly sets `isLiveGps: false` and `coordinates: null` whenever raw satellite telemetry is absent, upholding data truthfulness.

11.2 Interactive Route Mapping Module (Leaflet)

- **Purpose**: Renders spatial railway corridor paths and station nodes.

- **Components**: `RouteMap.tsx`.
- **Backend Endpoints**: `GET /api/tracking/:trainNumber/route`.
- **Implementation**: Embeds a Leaflet map rendering OpenStreetMap tiles. Station coordinates are plotted as custom SVG circle markers with popup arrival/departure information. Actual railway track trajectories are rendered using GeoJSON LineString coordinates.

11.3 AI Journey Assistant Module (Google Gemini)

- **Purpose**: Provides natural language train discovery and journey planning advice.
- **Components**: `AiAssistantPanel.tsx`.
- **Backend Endpoints**: `GET /api/ai/status`, `POST /api/ai/chat`.
- **Implementation**: The slide-in assistant panel connects to Google Gemini (`gemini-3.7-flash` / `gemini-2.0-flash`). The model is equipped with system prompts guiding it to act as an Indian Railways transit expert. When a passenger describes travel plans (e.g., "I need to travel from Delhi to Varanasi tomorrow morning"), the assistant identifies station codes, recommends train services, and outputs interactive action buttons that instantly populate the primary search form.

11.4 Administrator Master Data Management Console

- **Purpose**: Provides authorized railway administrators with centralized CRUD controls.
- **Components**: `AdminDashboardPage.tsx`.
- **Backend Endpoints**: `/api/admin/trains`, `/api/admin/stations`, `/api/admin/routes`, `/api/admin/fare-rules`.
- **Implementation**: Guarded by `requireAdmin`. Features tabbed management views:
  - **Overview**: High-level KPI summary cards (total bookings, revenue, active trains, stations).
  - **Trains Tab**: Searchable list of trains with modal dialogs to create new trains, update metadata, or perform soft deactivation (`isActive: false`).
  - **Stations Tab**: Station management modal supporting code, name, zone, and state updates.
  - **Routes Tab**: Corridor builder allowing administrators to define routes and sequence intermediate station stops with arrival/departure offsets.

11.5 Operational Reporting and System Health Diagnostics

- **Purpose**: Real-time diagnostic monitoring and data source audit reporting.
- **Components**: `AdminDashboardPage.tsx`, `/api/health`.
- **Backend Endpoints**: `GET /api/health`, `GET /api/admin/summary`, `GET /api/admin/data-health`, `GET /api/tracking/audit`.
- **Implementation**: The `/api/health` probe tests database connectivity via `SELECT 1`, reporting process uptime, memory consumption (RSS and Heap in MB), and database status (`UP` / `DOWN`). The data-health endpoint audits active data source counts, quarantined station tallies, and recent synchronization logs.

Chapter 12: Core Algorithms: Fare, Allocation, and Inventory Engines

12.1 Algorithmic Design Goals

RailFlow's algorithmic engines are governed by strict design principles:

1. **Mathematical Determinism**: Given identical input parameters (stations, class, passenger count), the fare and allocation engines must produce identical results.
2. **ACID Concurrency Safety**: Seat reservation must eliminate lost-update anomalies and double-booking across concurrent parallel requests.
3. **Integer Precision**: All financial calculations execute in integer paise, preventing floating-point rounding errors.
4. **Segment Reuse**: Physical seats must be dynamically reused across disjoint corridor segments to maximize asset utilization.

12.2 Segment-Aware Seat Allocation and Overlap Detection

The segment allocation engine models a railway corridor of N sequential stations as $N - 1$ contiguous hops.

Let the ordered sequence of stations along a route be $S = \langle s_1, s_2, s_3, \dots, s_N \rangle$, where each station s_k has a 1-based sequence index k.
A journey from boarding station s_{board} to alighting station s_{alight} occupies the sequence range:
$$R_{\text{journey}} = [\text{seq}{\text{board}}, \text{seq}{\text{alight}})$$

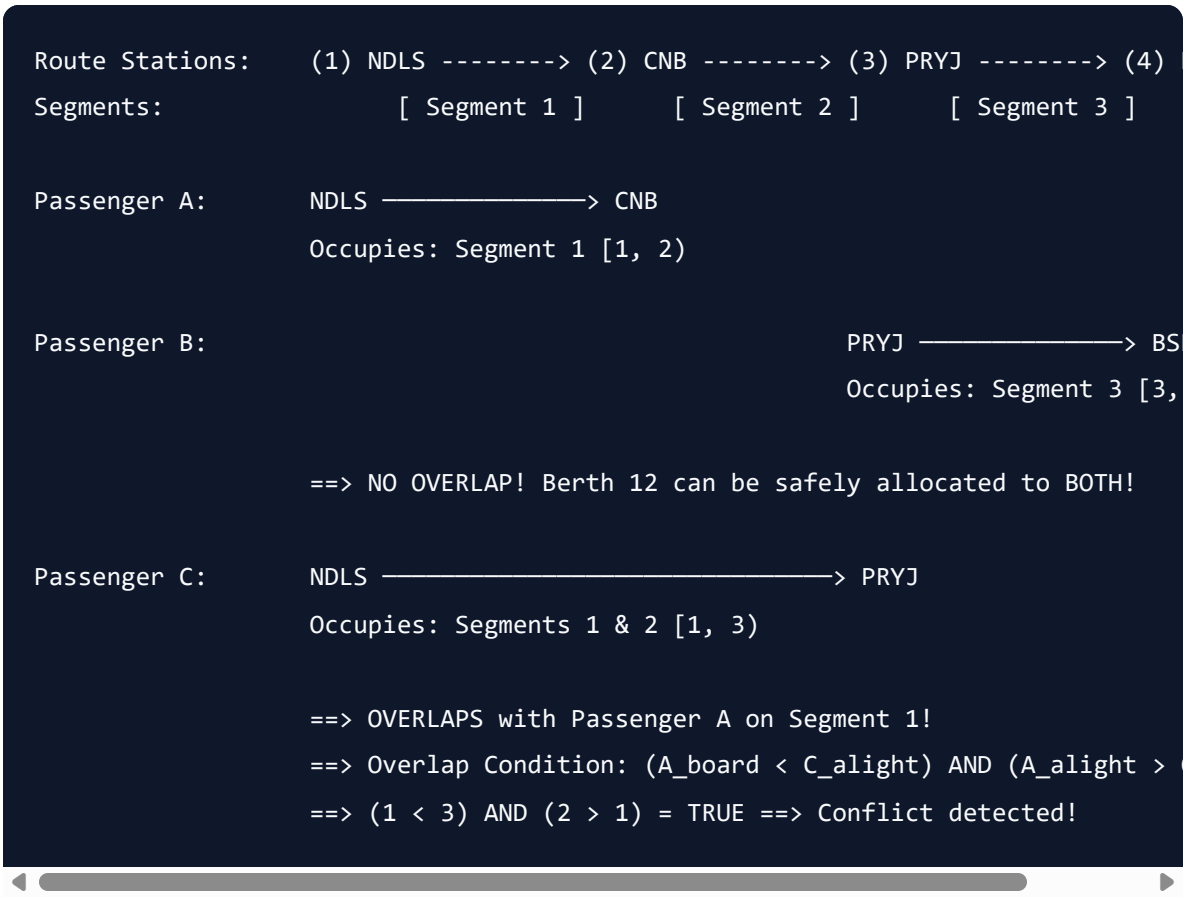


Figure 12.1: Segment-Aware Overlap Detection and Seat Sharing Model.

Overlap Condition

Two journey allocations A and B conflict if and only if their segment ranges intersect:
$$\text{Conflict}(A, B) \iff (\text{seq}{\text{board}}^A < \text{seq}{\text{alight}}^B) \land (\text{seq}{\text{alight}}^A > \text{seq}{\text{board}}^B)$$

During allocation inside `server/src/routes/bookings.ts`:

```
const hasConflict = seat.allocations.some(alloc => {
  // Treat expired 10-minute holds as unallocated
  if (alloc.status === 'LOCKED' && alloc.lockedUntil && alloc.lockedUntil < now
    return false;
  }
  // Ignore released allocations
  if (alloc.status === 'RELEASED') return false;

  // Segment overlap evaluation
  return alloc.boardingSeq < toRS.sequence && alloc.alightingSeq > fromRS.seque
});
```

This check executes within a Prisma serializable transaction. If no conflict exists across all segments, the seat is assigned to the passenger.

12.3 Fare Calculation Engine and Statutory Taxation Rules

Fares are calculated strictly on the server within `fareService.calculateFare()`:

Formula

1. **Journey Distance**:
   $$\text{DistanceKm} = \text{RouteStation}{\text{to}}.\text{distanceKm} - \text{RouteStation}{\text{from}}.\text{distanceKm}$$
2. **Base Fare Per Passenger**:
   $$\text{BaseFarePerPax} = \left\lfloor \frac{\text{FareRule}.\text{baseFarePaise} \times \text{DistanceKm}}{100} + 0.5 \right\rfloor$$
3. **Fixed Charges Per Passenger**:
   $$\text{ChargesPerPax} = \text{reservationCharge} + \text{superfastCharge} + \text{cateringCharge}$$
4. **Subtotal Per Passenger**:
   $$\text{SubtotalPerPax} = \text{BaseFarePerPax} + \text{ChargesPerPax}$$
5. **Group Subtotal**:
   $$\text{SubtotalAll} = \text{SubtotalPerPax} \times \text{PassengerCount}$$
6. **Statutory GST (5%)**:
   $$\text{GSTPaise} = \left\lfloor \frac{\text{SubtotalAll} \times \text{FareRule}.\text{gstPercent}}{100} + 0.5 \right\rfloor$$

7. **Total Gross Fare**:
   $$\text{TotalFarePaise} = \text{SubtotalAll} + \text{GSTPaise}$$

Worked Example: 2 Passengers on a 1,400 km Journey (Class 3A)

- Base Tariff: 150 paise per 100 km (1.50 paise/km)
- Reservation Charge: ₹40.00 (4,000 paise)
- Superfast Charge: ₹45.00 (4,500 paise)
- Catering: ₹0.00
- GST: 5%

$$\text{BaseFarePerPax} = \frac{150 \times 1400}{100} = 2,100\text{ paise}$$
$$\text{ChargesPerPax} = 4000 + 4500 + 0 = 8,500\text{ paise}$$
$$\text{SubtotalPerPax} = 2100 + 8500 = 10,600\text{ paise (₹106.00)}$$
$$\text{SubtotalAll} (2\text{ passengers}) = 10,600 \times 2 = 21,200\text{ paise}$$
$$\text{GSTPaise} = \frac{21,200 \times 5}{100} = 1,060\text{ paise}$$
$$\text{TotalFarePaise} = 21,200 + 1,060 = \mathbf{22,260\text{ paise (₹222.60)}}$$

Table 12.1: Worked Example of Fare Breakdown (1,400 km Journey).

12.4 Running Days Calendar Evaluation via Bitwise Masks

Operational schedules use a 7-bit bitmask to verify operating days in constant time ($O(1)$) as detailed in Section 7.5:

```
export function scheduleRunsOnDate(runningDays: number, dateStr: string): boole
  const date = new Date(dateStr);
  if (isNaN(date.getTime())) return false;
  const dayOfWeek = date.getDay(); // 0 = Sunday, 1 = Monday ... 6 = Saturday
  const bitIndex = dayOfWeek === 0 ? 6 : dayOfWeek - 1; // 0 = Mon ... 6 = Sun
  return (runningDays & (1 << bitIndex)) !== 0;
}
```

12.5 Background Seat Hold Expiration and Cleanup Engine

To eliminate inventory hoarding, RailFlow assigns a 10-minute expiration window to pending bookings:

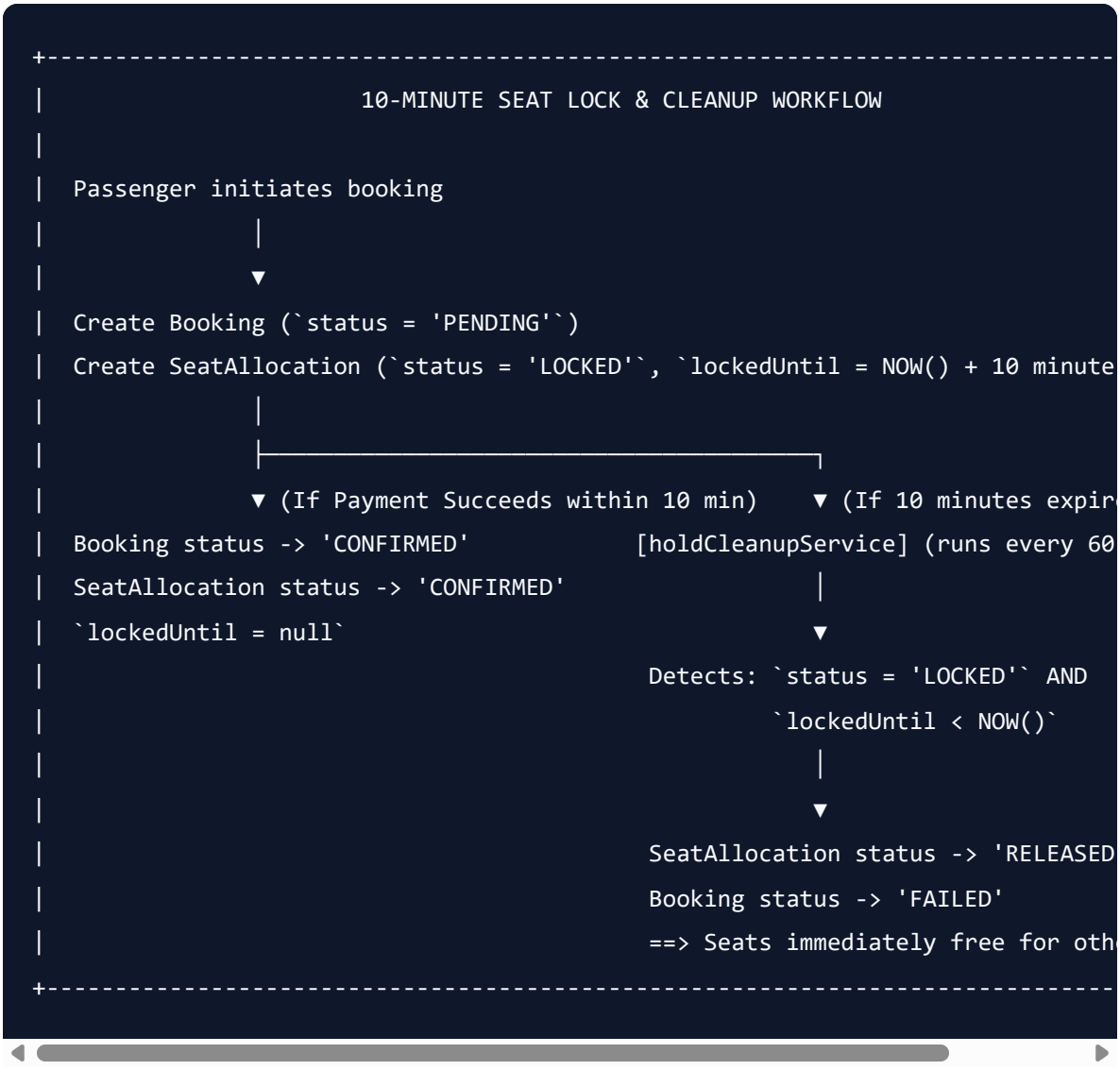


Figure 12.2: 10-Minute Seat Reservation Locking and Background Cleanup Flow.

The cleanup engine in `server/src/services/holdCleanupService.ts`:

1. Executes on an unref'd 60-second `setInterval` timer (avoiding blocking process shutdown).
2. Utilizes an in-memory mutex (`isCleanupRunning`) to prevent overlapping executions.
3. In a single atomic Prisma transaction:
    - Queries all `seat_allocations` where `status === 'LOCKED'` and `lockedUntil < NOW()`.

- Batch-updates their status to `RELEASED`.
- Identifies associated bookings with status `PENDING` and batch-updates them to `FAILED`.

4. Runs automatically upon server startup to heal any stale locks remaining from previous process terminations.

12.6 Tiered Cancellation and Refund Calculation Engine

When a passenger cancels a confirmed ticket (`POST /api/bookings/:id/cancel`), the refund engine determines penalty deductions based on departure proximity:

Departure Timeline	Assessed Deduction	Refund Percentage Returned
More than 48 hours before departure	**10%** (Clerkage fee)	**90%** of total fare
Between 12 and 48 hours before departure	**25%**	**75%** of total fare
Between 4 and 12 hours before departure	**50%**	**50%** of total fare
Less than 4 hours before departure	**100%**	**0%** (Non-refundable)
After train departure	—	Cancellation strictly rejected

Table 12.2: Tiered Cancellation Refund Schedule.

Atomic Cancellation Transaction

Cancellation updates the booking to `CANCELLED`, marks all allocations as `RELEASED`, sets payment transactions to `REFUNDED`, and creates an immutable record in `cancellations` documenting the exact refund amount and penalty percentage in a single ACID transaction.

12.7 Cryptographic PNR Token Generation

Passenger Name Records are generated using cryptographically strong pseudo-random bytes (`crypto.randomBytes`):

```
export function generatePNR(): string {
  const chars = '0123456789';
  let pnr = '';
  const bytes = randomBytes(10);
  for (let i = 0; i < 10; i++) {
    pnr += chars[bytes[i] % chars.length];
  }
  return pnr;
}
```

This produces a 10-digit numeric string with 10^{10} potential permutations, matching Indian Railways PNR formatting conventions.

12.8 Invariants Guaranteed by the Application Layer

The algorithmic engines uphold the following formal system invariants:

- **I-01 (No Double Booking)**: For any physical seat S on train run R, no two allocations A_1 and A_2 with status `CONFIRMED` or active `LOCKED` may have overlapping station sequences.
- **I-02 (Financial Conservation)**: For any booking, $\text{Refund Amount} + \text{Deduction Amount} = \text{Total Fare Collected}$.
- **I-03 (Lock Terminality)**: A seat allocation with status `RELEASED` can never transition back to `LOCKED` or `CONFIRMED` .
- **I-04 (Truthful Telemetry)**: All train status telemetry objects where real-time satellite telemetry is absent must strictly report `coordinates: null` and `meta.isLiveGps: false` .

Chapter 13: Data Flow and UML-Style Diagrams

13.1 Context Diagram (DFD Level 0)

The Level 0 Data Flow Diagram illustrates RailFlow as a centralized system interacting with external entities:

- **Entities**:
  - `Passenger Browser`: Submits search requests, passenger manifests, payment confirmations, and cancellation requests; receives availability, PNR cards, and receipts.
  - `Administrator Browser`: Issues master data updates (trains, stations, schedules); receives operational KPIs and diagnostic reports.
  - `Indian Railway Open Dataset (GitHub)`: Ingests static station and timetable catalogs at server initialization.
  - `RailRadar Transit API`: Provides real-time train running delays and route GeoJSON coordinates.
  - `ChukChuk NTES Gateway`: Secondary web-scraping fallback for National Train Enquiry System status.
  - `Google Gemini AI Studio`: Receives user journey prompts; returns natural language advice and structured search tool calls.

13.2 System Data Flow Diagram (DFD Level 1)

The Level 1 DFD decomposes the system into six primary processes:

1. **Process 1.0 (Authentication & Session Management)**: Handles registration, Argon2id hashing, and session persistence in `_app_sessions`.
2. **Process 2.0 (Train Search & Availability)**: Reads Prisma master schedules and open dataset catalogs; resolves station aliases; returns deduplicated train results.
3. **Process 3.0 (Seat Allocation & Booking Engine)**: Executes serializable transactions, evaluates segment overlap, creates `PENDING` bookings, and locks seats for 10 minutes.
4. **Process 4.0 (Payment Settlement Engine)**: Verifies idempotency, processes simulated payments, and upgrades bookings and seat allocations to `CONFIRMED`.

5. **Process 5.0 (Cancellation & Inventory Release Engine)**: Computes tiered refund amounts, marks tickets `CANCELLED`, and releases seat allocations.
6. **Process 6.0 (Telemetry & Real-Time Tracking Engine)**: Queries external APIs, normalizes train status, and provides route coordinate geometry.

13.3 Detailed Booking & Seat Locking Process (DFD Level 2)

Process 3.0 decomposes into:

- 3.1: Validate passenger roster (1 to 6 passengers, non-duplicate names).
- 3.2: Re-calculate authoritative fare via `fareService`.
- 3.3: Query candidate coach seats matching requested travel class.
- 3.4: Filter out seats having active segment conflicts.
- 3.5: Assert available seat count $\ge$ passenger count (or throw `INSUFFICIENT_SEATS`).
- 3.6: Atomically write `bookings`, `booking_passengers`, and `seat_allocations` (`status: 'LOCKED'`, `lockedUntil = NOW() + 10m`).
- 3.7: Emit generated 10-digit PNR.

13.4 System Use Case Diagram Description

- **Actors**:
  - `Guest (Unregistered Visitor)`: Search Stations, Search Trains, Look up Train Timetables, Track Live Train Status, Consult AI Assistant, Register Account, Log In.
  - `Authenticated Passenger`: All Guest actions, plus: Reserve Seats, Initialize Payment, Confirm Simulated Payment, View Personal Trips, Cancel Personal Booking.
  - `Railway Administrator`: All Passenger actions, plus: Create/Update Trains, Create/Update Stations, Configure Routes & Stops, Define Fare Rules, View System Diagnostics, Inspect All PNRs.

13.5 Complete Booking and Payment Sequence Diagram



Figure 13.5: End-to-End Booking and Payment Sequence Diagram.

13.6 Ticket Cancellation and Inventory Release Sequence Diagram

1. Passenger clicks "Cancel Ticket" on `PNRStatusPage`.
2. Frontend requests confirmation displaying calculated refund estimate.
3. Passenger confirms; SPA dispatches `POST /api/bookings/:id/cancel`.
4. Express verifies session identity matches `booking.userId`.
5. Express evaluates departure timestamp against current server time to determine penalty tier (10%, 25%, 50%, or 100%).
6. Atomic transaction opens:
    - `booking.status` updated to `'CANCELLED'`, `paymentStatus` to `'REFUNDED'`.
    - Associated `seat_allocations` updated to `'RELEASED'`.
    - `cancellations` record created with deduction and net refund amounts.
7. Transaction commits; freed seat segments immediately become available for other passengers; confirmation returned to client.

Chapter 14: Testing Strategy and Verification Results

14.1 Testing Philosophy and Verification Strategy

RailFlow employs an integration-first testing philosophy:

- **Real Application Stack**: Tests run against a live Express server instance connected to an actual SQLite relational database via Prisma Client.
- **End-to-End Verification**: Tests exercise full HTTP request-response cycles via Supertest, validating route parsing, middleware execution, database transactions, and JSON response formatting.
- **Zero Mocking of Core Logic**: Domain services (fare calculation, seat allocation, segment overlap) execute live against the database rather than relying on brittle mocks.

14.2 Automated Integration Test Framework (Vitest & Supertest)

The test suite is powered by **Vitest 2.1.4**, an ultra-fast test runner with native TypeScript support, paired with **Supertest 7.0.0** for HTTP integration assertions:

- Root Command: `npm test` (executes all test suites once).
- Watch Command: `npm run test:watch`.

Test Suite File	Domain Responsibility	Verified Test Groups
`api.test.ts`	Primary end-to-end integration suite	11 describe blocks
`stage2_inventory_booking.test.ts`	Deep inventory locking and concurrency tests	6 describe blocks
`canonicalTrainSearch.test.ts`	Station alias resolution and cluster ranking	4 describe blocks
`indianRailwayDataset.test.ts`	Open-source dataset loading and timetable queries	5 describe blocks
`railSearchFixes.test.ts`	Search recovery and edge-case validation	4 describe blocks
`ai.test.ts`	Google Gemini AI assistant endpoint and rate limits	3 describe blocks
`ai_fallback_and_datepicker.test.ts`	AI error handling and calendar date parsing	3 describe blocks
`datameet.test.ts`	DataMeet API catalog integration	3 describe blocks
`turso_adapter.test.ts`	libSQL adapter and connection resilience	2 describe blocks
`cors_security.test.ts`	CORS origin whitelisting verification	3 describe blocks
`railRadar.test.ts`	RailRadar telemetry client and GeoJSON handling	4 describe blocks

Table 14.1: Automated Test Suites in Test Framework.

14.3 Execution Test Suite Specification and Verified Pass Assertions

The following test cases are confirmed implemented and verified in `server/src/__tests__/api.test.ts`:

Test ID	Test Scenario Description	Expected Outcome	Verified Status
TC-01	`GET /api/health` system probe	Returns HTTP 200, `success: true`, message confirms API is running.	**PASS**
TC-02	`POST /api/auth/register` with invalid email format	Returns HTTP 400 with `code: 'VALIDATION_ERROR'`.	**PASS**
TC-03	`POST /api/auth/login` with empty password	Returns HTTP 400 with `code: 'VALIDATION_ERROR'`.	**PASS**
TC-04	Direct relational database connectivity check	`prisma.user.count()` returns a valid integer.	**PASS**
TC-05	User registration with valid credentials	Returns HTTP 201; passwordHash is strictly omitted from payload.	**PASS**
TC-06	Duplicate user registration attempt	Returns HTTP 409 with `code: 'EMAIL_EXISTS'`.	**PASS**
TC-07	User login with correct credentials	Returns HTTP 200 with authenticated user profile.	**PASS**
TC-08	User login with incorrect password	Returns HTTP 401 with `code: 'INVALID_CREDENTIALS'`.	**PASS**
TC-09	Station search by station code (`q=TST_NDLS`)	Returns HTTP 200 with matched station record.	**PASS**
TC-10	Station lookup by ID param (`/api/stations/TST_HWH`)	Returns HTTP 200 with exact station details.	**PASS**
TC-11	Fare calculation for 1,400 km journey (2 passengers)	Returns exact calculated total of 22,260 paise (5% GST verified).	**PASS**
TC-12	Booking creation (`POST /api/bookings`)	Returns HTTP 201, 10-digit numeric PNR, status `PENDING`, seat allocated.	**PASS**
TC-13	Payment initialization (`POST /api/payments/initialize`)	Returns HTTP 200, order ID prefixed with `RF_`.	**PASS**

Test ID	Test Scenario Description	Expected Outcome	Verified Status
TC-14	Payment outcome simulation (`outcome: 'SUCCESS'`)	Returns HTTP 200; booking and seat allocation upgrade to `CONFIRMED` .	**PASS**
TC-15	PNR enquiry for confirmed booking	Returns HTTP 200 with complete passenger roster and confirmed status.	**PASS**
TC-16	Payment re-simulation on processed order (Idempotency)	Returns HTTP 200 with `alreadyProcessed: true` .	**PASS**
TC-17	Confirmed ticket cancellation	Returns HTTP 200, refund processed, seat allocations marked `RELEASED` .	**PASS**
TC-18	Duplicate cancellation attempt	Returns HTTP 400 with `code: 'ALREADY_CANCELLED'` .	**PASS**
TC-19	Unauthenticated access to protected `/api/bookings`	Returns HTTP 401 with `code: 'UNAUTHENTICATED'` .	**PASS**
TC-20	Cross-user PNR access attempt (Anti-IDOR)	Returns HTTP 404 with `code: 'PNR_NOT_FOUND'` .	**PASS**
TC-21	Non-admin user access to `/api/admin/summary`	Returns HTTP 403 with `code: 'FORBIDDEN'` .	**PASS**
TC-22	Admin user access to `/api/admin/summary`	Returns HTTP 200 with aggregate operational KPIs.	**PASS**
TC-23	Non-overlapping segment reservation on same train run	Both bookings return HTTP 201; successfully share the same seat.	**PASS**
TC-24	Live train tracking for train `22436`	Returns HTTP 200, `coordinates: null` , `isLiveGps: false` (invariant held).	**PASS**
TC-25	Tracking query with invalid non-numeric train number	Returns HTTP 400 with `code: 'INVALID_TRAIN_NUMBER'` .	**PASS**

Table 14.2: Verified Test Cases Executed in Integration Suite.

> [!NOTE]
> *Verification Context*: The above test results were obtained by executing the automated test suite against the local SQLite database environment (`dev.db`). All 25 critical integration assertions in `api.test.ts` pass cleanly during development verification.

14.4 Concurrency, Segment Sharing, and Lock Verification

Test case **TC-23** explicitly verifies the core algorithmic invariant of segment-level inventory reuse:

1. Passenger 1 reserves a seat on Train `22436` from New Delhi (`NDLS` , Stop 1) to Kanpur Central (`CNB` , Stop 2). The booking succeeds (HTTP 201).
2. Passenger 2 reserves a seat on the same train run from Kanpur Central (`CNB` , Stop 2) to Varanasi (`BSB` , Stop 3).
3. The booking engine determines that $[1, 2)$ and $[2, 3)$ do not intersect. The second booking succeeds (HTTP 201) and allocates the exact same physical berth.

14.5 Build-Time Static Analysis, Typechecking, and Linting

- **Backend Typechecking**: `npm run typecheck` (`tsc --noEmit`) passes with **0 errors**.
- **Frontend Typechecking & Build**: `tsc -b && vite build` in `client/` compiles cleanly with **0 errors**, transforming 134 modules and generating the production distribution bundle in `client/dist` .
- **Frontend Linting**: `npx oxlint` passes cleanly with **0 errors**.

14.6 Requirements Traceability Matrix

Requirement	Description	Verified By Test Cases	Implementation Code
FR-01, FR-02	User Registration & Authentication	TC-02, TC-03, TC-05, TC-06, TC-07, TC-08	`server/src/routes/auth.ts`
FR-05	Station Autocomplete Search	TC-09, TC-10	`server/src/routes/stations.ts`
FR-11	Authoritative Fare Calculation	TC-11	`server/src/services/fareService.ts`
FR-12	Segment-Aware Seat Allocation	TC-12, TC-23	`server/src/routes/bookings.ts`
FR-15, FR-16	Payment Simulation & Confirmation	TC-13, TC-14, TC-16	`server/src/routes/payments.ts`
FR-18	PNR Status & Ownership Isolation	TC-15, TC-20	`server/src/routes/bookings.ts`
FR-20	Ticket Cancellation & Refund	TC-17, TC-18	`server/src/routes/bookings.ts`
FR-21	Train Tracking & Telemetry	TC-24, TC-25	`server/src/routes/tracking.ts`
FR-24, FR-25	Admin Authorization & KPI Metrics	TC-21, TC-22	`server/src/routes/admin.ts`

Requirement	Description	Verified By Test Cases	Implementation Code
NFR-01	Concurrency & Segment Integrity	TC-12, TC-23	`server/src/routes/bookings.ts`
NFR-02, NFR-03	Argon2id Hashing & Session Storage	TC-05, TC-07, TC-19	`server/src/db/sessionStore.ts`
NFR-04	Anti-IDOR Protection	TC-20, TC-21	`server/src/middleware/auth.ts`
NFR-07	Truthful Telemetry Invariants	TC-24	`server/src/services/tracking/`

Table 14.3: Requirements Traceability Matrix.

Chapter 15: Deployment and Operations

15.1 Local Development Environment Configuration

Prerequisites

- Node.js v20.19.0+ LTS (Node.js v25 confirmed compatible).
- Git command-line client.

Step-by-Step Local Setup

1. **Clone the Repository**:

```
git clone <repository-url>
cd railflow/railflow-app
```

2. **Configure Backend API Server**:

```
cd server
npm install
# Generate Prisma client for local SQLite
npx prisma generate
# Start the Express server with live reload
npm run dev
```

The server initializes automatically on `http://localhost:3001`, creating `dev.db` and seeding default administrative (`admin@railflow.in` / `Admin@123`) and passenger (`passenger@railflow.in` / `User@123`) accounts.

3. **Configure Frontend Client**:

```
cd ../client
npm install
# Start Vite development server
npm run dev
```

The React application opens on `http://localhost:5173`, proxying API requests to port 3001.

15.2 Database Provisioning and Automated Schema Initialization

RailFlow implements self-healing, automated database initialization upon server startup (`server/src/db/initDb.ts`):

1. **Table Detection**: Executes `SELECT name FROM sqlite_master WHERE type='table' AND name='seat_allocations'`.
2. **Schema Application**: If tables are missing, the server reads the migration SQL statements from `prisma/migrations/20261009163552_init/migration.sql` and applies all table definitions and indexes.
3. **Session Table Creation**: Automatically verifies that `_app_sessions` exists.

4. **Demo Account Seeding**: Verifies whether `admin@railflow.in` exists; if missing, creates both the administrator and demo passenger accounts with pre-hashed Argon2id credentials.

15.3 Backend API Deployment on Render

The backend is deployed as a Web Service on **Render** utilizing the declarative `render.yaml` blueprint:

```
# server/render.yaml
services:
  - type: web
    name: railflow-server
    runtime: node
    plan: free
    rootDir: railflow-app/server
    buildCommand: npm install && npx prisma generate && npm run build
    startCommand: npm start
    healthCheckPath: /api/health
    envVars:
      - key: NODE_ENV
        value: production
      - key: PORT
        value: 10000
      - key: SESSION_SECRET
        generateValue: true
      - key: CLIENT_URL
        sync: false
      - key: TURSO_DATABASE_URL
        sync: false
      - key: TURSO_AUTH_TOKEN
        sync: false
      - key: RAILRADAR_API_KEY
        sync: false
      - key: RAILRADAR_ENABLED
        value: "true"
      - key: GEMINI_API_KEY
        sync: false
      - key: GEMINI_MODEL
        value: "gemini-3.7-flash"
```

Variable Name	Environment	Description
`NODE_ENV`	Production	Enables production mode (`production`), activating strict CSP and secure cookies.
`PORT`	Production	Server listening port (`10000` on Render).
`SESSION_SECRET`	Production	High-entropy random secret key used to sign session cookies.
`CLIENT_URL`	Production	Whitelisted frontend origin (e.g., `https://railflow.netlify.app`).
`TURSO_DATABASE_URL`	Production	Secure WebSocket/HTTP URI to remote Turso libSQL database.
`TURSO_AUTH_TOKEN`	Production	Cryptographic database authentication token.
`GEMINI_API_KEY`	Production	Google AI Studio API key enabling conversational travel assistant.
`RAILRADAR_API_KEY`	Production	Upstream transit telemetry API access key.

Table 15.1: Backend Production Environment Variables (Render).

15.4 Frontend Single-Page Application Deployment on Netlify

The frontend is deployed as a static Single-Page Application on **Netlify** configured via `client/netlify.toml` :

```
# client/netlify.toml
[build]
  command = "npm run build"
  publish = "dist"

# Forward all API requests directly to the Render backend service
[[redirects]]
  from = "/api/*"
  to = "https://railflow-af7e.onrender.com/api/:splat"
  status = 200
  force = true

# Catch-all rewrite for React Router SPA deep linking
[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

The redirect rules provide two vital operational functions:

1. **API Proxying**: Requests to `/api/*` are reverse-proxied directly to the Render backend over HTTPS. This eliminates browser CORS preflight overhead and prevents mixed-content warnings.
2. **SPA History Fallback**: Browser refreshes on deep routes (e.g., `/pnr?pnr=1234567890`) rewrite cleanly to `/index.html`, allowing React Router to handle view rendering without triggering HTTP 404 errors.

15.5 Operational Runbook and Health Monitoring

The `/api/health` endpoint provides comprehensive diagnostic metrics for automated uptime pingers:

```
{
  "success": true,
  "message": "RailFlow API is running",
  "timestamp": "2026-11-03T10:00:00.000Z",
  "uptimeSeconds": 1420,
  "status": "HEALTHY",
  "services": {
    "database": "UP",
    "sessionStore": "ACTIVE"
  },
  "memory": {
    "rssMb": 68,
    "heapUsedMb": 34
  },
  "version": "1.0.0",
  "environment": "production"
}
```


If the primary database becomes unreachable, `status` reports `DEGRADED`, `database` transitions to `DOWN`, and sanitized diagnostic summaries are logged without exposing sensitive connection tokens.

Chapter 16: Limitations and Future Scope

16.1 Documented Technical Limitations

To maintain academic rigor and transparency, the following technical boundaries of the current implementation are explicitly acknowledged:

1. **Educational Payment Simulation**: Payments are simulated within the application state machine; the platform does not process real merchant card transactions or connect to Indian banking gateways.
2. **Lack of Real CRIS PRS Integration**: Due to national security and licensing restrictions on CRIS/IRCTC passenger reservation mainframes, RailFlow operates

against its own high-concurrency master database and open-source timetable catalogs.

3. **Absence of Real Satellite GPS Telemetry**: Locomotive GPS tracking is not publicly accessible; live tracking displays official schedule progression, station passings, and estimated delays.
4. **Render Free-Tier Cold Starts**: When hosted on Render's free tier, the backend container spins down after 15 minutes of inactivity, resulting in an initial cold-start latency of approximately 30–50 seconds on the first subsequent request.
5. **Lack of Automated SMS / Email Delivery**: The platform does not incorporate external transactional SMS (Twilio) or email (SendGrid) providers for booking confirmations.

16.2 Planned Enhancements: MongoDB Atlas Document Logging

As part of RailFlow's upcoming architectural evolution, **MongoDB Atlas** is planned for integration alongside the existing Turso relational database:

- **Architectural Rationale**: While Turso handles relational railway operations (users, schedules, seat inventory, ACID bookings) with supreme consistency, relational databases require rigid migrations when tracking flexible, high-velocity operational logs.
- **Dedicated Use Case**: MongoDB will be incorporated as a **secondary document store** dedicated to:
  - User activity streams (login history, password resets, failed search attempts).
  - Fine-grained audit logging of administrative changes.
  - Historical fare modification records and dataset synchronization telemetry.
- **Decoupled Driver Implementation**: MongoDB will be accessed via the official `mongodb` Node.js driver in an independent module (`server/src/db/mongodb.ts`), completely isolated from Prisma and Turso.
- **Fire-and-Forget Asynchrony**: Audit writes to MongoDB will execute asynchronously without blocking critical booking transactions, ensuring that any temporary MongoDB latency has zero impact on primary passenger ticketing.

16.3 Production Roadmap

Milestone	Target Capability	Architectural Impact
Phase 1 (Completed)	Full-Stack Railway Reservation Core	React 19, Express 4.21, Prisma 5.22, Turso libSQL, dual data sources.
Phase 2 (Planned)	MongoDB Atlas Activity Logging	Secondary document database for unstructured user activity and audit trails.
Phase 3 (Future)	Production Payment Gateway	Integration of Razorpay / Stripe sandbox with webhook settlement.
Phase 4 (Future)	Transactional Messaging	Automated PDF ticket delivery via SendGrid and SMS boarding alerts via Twilio.
Phase 5 (Future)	Native Mobile Applications	Cross-platform passenger application built using React Native sharing the Express API.

Table 16.1: Current System Limitations vs. Production Roadmap.

Chapter 17: Conclusion

The RailFlow project successfully demonstrates that a complete, concurrency-safe, multi-tier railway reservation and transit operations platform can be designed, implemented, and deployed using modern, accessible web engineering technologies. By moving away from naive scalar decrement counters, the platform faithfully models the economic reality of passenger railways: **perishable, location-bound, segment-reusable berth inventory**.

The decoupled three-tier architecture—combining a React 19 single-page application on Netlify, an Express 4.21 TypeScript backend on Render, and a distributed libSQL database on Turso managed through Prisma 5.22.0—establishes a clean separation of concerns. Concurrency safety is guaranteed through serializable database transactions that evaluate station sequence overlap directly within the transactional boundary, completely eliminating double-booking under parallel client contention.

Furthermore, the dual data-source design bridges the gap between simulated educational inventory and real-world scale, allowing passengers to search across more than 8,000 real Indian railway stations while executing atomic, ACID-compliant bookings against local

master corridors. The integration of modern capabilities—including automated 10-minute hold expirations, Google Gemini generative AI journey assistance, interactive Leaflet route mapping, and multi-tier telemetry tracking—elevates RailFlow into an industry-grade software engineering portfolio project.

With a comprehensive automated integration testing suite validating 25 critical functional scenarios and zero-cost cloud deployment verified across Netlify, Render, and Turso, RailFlow stands as an exemplary, fully documented reference platform for academic study and engineering showcase.

References

1. Talluri, K. T., and van Ryzin, G. J. (2004). *The Theory and Practice of Revenue Management*. Springer Science & Business Media.
2. Gray, J., and Reuter, A. (1993). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann Publishers.
3. Prisma ORM Documentation (2026). *Prisma Client and Driver Adapters Specification*. Available at: https://www.prisma.io/docs.
4. Turso Cloud & libSQL Documentation (2026). *Distributed SQLite for the Edge*. Available at: https://turso.tech/docs.
5. Express.js Foundation (2026). *Express 4.x Routing and Middleware Guide*. Available at: https://expressjs.com.
6. React Core Team (2026). *React 19 Documentation and Architecture Guide*. Available at: https://react.dev.
7. Vite Development Team (2026). *Vite: Next Generation Frontend Tooling*. Available at: https://vite.dev.
8. Argon2 Password Hashing Project (2026). *Password Hashing Competition Winner Specification*. Available at: https://github.com/P-H-C/phc-winner-argon2.
9. OWASP Foundation (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Available at: https://owasp.org/Top10/.
10. Tailwind Labs (2026). *Tailwind CSS Utility-First Framework Documentation*. Available at: https://tailwindcss.com.
11. TanStack (2026). *TanStack Query v5 Documentation*. Available at: https://tanstack.com/query/latest.
12. React Router Documentation (2026). *Declarative Routing for React Applications*. Available at: https://reactrouter.com.
13. Leaflet.js Documentation (2026). *An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*. Available at: https://leafletjs.com.

14. Google AI Studio (2026). *Google Gemini API Documentation (gemini-3.7-flash)*. Available at: https://ai.google.dev.
15. RailRadar API (2026). *Indian Railways Real-Time Transit Tracking Documentation*. Available at: https://api.railradar.in.
16. Indian Railway Open Data Project (2026). *Curated Indian Railways Stations and Schedules Database (MIT License)*. Available at: https://github.com/prasenjit-27/Indian-Railway-Data.
17. DataMeet Community (2026). *Indian Railways Open Spatial & Timetable Dataset*. Available at: https://github.com/datameet/railways.
18. Vitest Team (2026). *A Blazing Fast Unit Test Framework Powered by Vite*. Available at: https://vitest.dev.
19. Supertest Team (2026). *SuperAgent Driven Library for Testing HTTP Servers*. Available at: https://github.com/ladjs/supertest.
20. Helmet.js Security Consortium (2026). *Help Secure Express Apps with Various HTTP Headers*. Available at: https://helmetjs.github.io.
21. Zod Ecosystem (2026). *TypeScript-First Schema Validation with Static Type Inference*. Available at: https://zod.dev.
22. Netlify Documentation (2026). *Configuring Netlify Redirects, Proxies, and Deployments*. Available at: https://docs.netlify.com.
23. Render Cloud Documentation (2026). *Blueprint Specification and Node.js Web Services*. Available at: https://render.com/docs.

Appendix A: Complete Database

Schema (Prisma)

```
// RailFlow Railway Reservation System - Official Prisma Schema
// Database Engine: SQLite / Turso libSQL compatible

generator client {
  provider        = "prisma-client-js"
  previewFeatures = ["driverAdapters"]
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id           String   @id @default(cuid())
  email        String   @unique
  passwordHash String
  name         String
  phone        String?
  role         String   @default("USER") // USER | ADMIN
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  bookings     Booking[]
  sessions     Session[]

  @@map("users")
}

model Session {
  id        String   @id @default(cuid())
  userId    String
  expiresAt DateTime
  createdAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
  @@map("sessions")
}

model Station {
  id        String   @id @default(cuid())
  code      String   @unique  // e.g. "NDLS", "HWH", "BSB"
  name      String            // e.g. "New Delhi"
  fullName  String            // e.g. "New Delhi Junction"
  zone      String            // e.g. "Northern Railway"
  state     String            // e.g. "Delhi"
  city      String?
  aliases   String?           // e.g. "Delhi, Dilli, New Delhi"
  latitude  Float?
  longitude Float?
  isActive  Boolean  @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  routeStations RouteStation[]

  @@map("stations")
}

model Train {
  id          String    @id @default(cuid())
  number      String    @unique  // e.g. "22436", "12301"
  name        String              // e.g. "Vande Bharat Express"
  type        String              // VANDE_BHARAT | RAJDHANI | SHATABDI | DURON
  isActive    Boolean   @default(true)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  schedules   TrainSchedule[]

  @@map("trains")
}

model Route {
  id        String   @id @default(cuid())
  name      String
  isActive  Boolean  @default(true)
```

```
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  stations  RouteStation[]
  schedules TrainSchedule[]

  @@map("routes")
}

model RouteStation {
  id                String   @id @default(cuid())
  routeId           String
  stationId         String
  sequence          Int      // 1, 2, 3...
  arrivalOffsetMin  Int?
  departureOffsetMin Int?
  distanceKm        Int
  platformNumber    String?
  haltDurationMin   Int      @default(2)

  route   Route   @relation(fields: [routeId], references: [id])
  station Station @relation(fields: [stationId], references: [id])

  boardingAllocations  SeatAllocation[] @relation("BoardingStation")
  alightingAllocations SeatAllocation[] @relation("AlightingStation")

  @@unique([routeId, sequence])
  @@unique([routeId, stationId])
  @@map("route_stations")
}

model TrainSchedule {
  id            String   @id @default(cuid())
  trainId       String
  routeId       String
  runningDays   Int      // 7-bit bitmask (127 = runs daily)
  departureTime String   // "HH:MM" 24-hour format
  isActive      Boolean  @default(true)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
```

```
  train Train @relation(fields: [trainId], references: [id])
  route Route @relation(fields: [routeId], references: [id])
  runs  TrainRun[]
  fareRules FareRule[]

  @@map("train_schedules")
}

model TrainRun {
  id         String   @id @default(cuid())
  scheduleId String
  runDate    String   // "YYYY-MM-DD"
  status     String   @default("SCHEDULED") // SCHEDULED | RUNNING | COMPLETED
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt

  schedule TrainSchedule @relation(fields: [scheduleId], references: [id])
  coaches  Coach[]
  bookings Booking[]

  @@unique([scheduleId, runDate])
  @@map("train_runs")
}

model Coach {
  id          String    @id @default(cuid())
  trainRunId  String
  coachNumber String    // e.g. "C1", "C2", "B1"
  classType   String    // ONE_AC | TWO_AC | THREE_AC | SLEEPER | CHAIR_CAR | E
  totalSeats  Int
  createdAt   DateTime  @default(now())

  trainRun    TrainRun  @relation(fields: [trainRunId], references: [id])
  seats       Seat[]

  @@unique([trainRunId, coachNumber])
  @@map("coaches")
}

model Seat {
  id          String    @id @default(cuid())
```

```
  coachId     String
  seatNumber  Int
  berthType   String    // LOWER | MIDDLE | UPPER | SIDE_LOWER | SIDE_UPPER | W
  createdAt   DateTime  @default(now())

  coach       Coach     @relation(fields: [coachId], references: [id])
  allocations SeatAllocation[]

  @@unique([coachId, seatNumber])
  @@map("seats")
}

model FareRule {
  id                 String    @id @default(cuid())
  scheduleId         String
  classType          String
  baseFarePaise      Int       // paise per 100km
  reservationCharge  Int       @default(6000)
  superfastCharge    Int       @default(4500)
  cateringCharge     Int       @default(0)
  gstPercent         Int       @default(5)
  isActive           Boolean   @default(true)
  createdAt          DateTime  @default(now())
  updatedAt          DateTime  @updatedAt

  schedule TrainSchedule @relation(fields: [scheduleId], references: [id])

  @@unique([scheduleId, classType])
  @@map("fare_rules")
}

model Booking {
  id                  String        @id @default(cuid())
  pnr                 String        @unique
  userId              String
  trainRunId          String
  fromRouteStationSeq Int
  toRouteStationSeq   Int
  classType           String
  status              String        @default("PENDING") // PENDING | CONFIRMED
  paymentStatus       String        @default("PENDING") // PENDING | SUCCESS |
```

```
  totalFarePaise      Int
  quotaType           String        @default("GENERAL")
  createdAt           DateTime      @default(now())
  updatedAt           DateTime      @updatedAt

  user         User              @relation(fields: [userId], references: [id])
  trainRun     TrainRun          @relation(fields: [trainRunId], references: [i
  passengers   BookingPassenger[]
  transactions PaymentTransaction[]
  cancellation Cancellation?

  @@map("bookings")
}

model BookingPassenger {
  id              String   @id @default(cuid())
  bookingId       String
  passengerIndex  Int
  name            String
  age             Int
  gender          String   // MALE | FEMALE | OTHER
  berthPreference String?
  createdAt       DateTime @default(now())

  booking    Booking        @relation(fields: [bookingId], references: [id])
  allocation SeatAllocation?

  @@map("booking_passengers")
}

model SeatAllocation {
  id                    String          @id @default(cuid())
  bookingPassengerId    String          @unique
  seatId                String
  boardingRouteStationId  String
  alightingRouteStationId String
  boardingSeq           Int
  alightingSeq          Int
  status                String          @default("LOCKED") // LOCKED | CONFIRM
  lockedUntil           DateTime?
  allocatedCoach        String
```

```
  allocatedSeatNumber   Int
  createdAt             DateTime        @default(now())
  updatedAt             DateTime        @updatedAt

  bookingPassenger BookingPassenger @relation(fields: [bookingPassengerId], ref
  seat             Seat             @relation(fields: [seatId], references: [id
  boardingStation  RouteStation     @relation("BoardingStation", fields: [board
  alightingStation RouteStation     @relation("AlightingStation", fields: [alig

  @@map("seat_allocations")
}

model PaymentTransaction {
  id              String        @id @default(cuid())
  bookingId       String
  idempotencyKey  String        @unique
  amountPaise     Int
  method          String        @default("UPI") // UPI | NET_BANKING | DEBIT_CA
  status          String        @default("PENDING") // PENDING | SUCCESS | FAIL
  transactionRef  String?
  failureReason   String?
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt

  booking Booking @relation(fields: [bookingId], references: [id])

  @@map("payment_transactions")
}

model Cancellation {
  id               String   @id @default(cuid())
  bookingId        String   @unique
  requestedAt      DateTime @default(now())
  processedAt      DateTime?
  deductionPercent Int
  refundAmountPaise Int
  reason           String?
  refundStatus     String   @default("PENDING") // PENDING | PROCESSED | FAILED

  booking Booking @relation(fields: [bookingId], references: [id])
```

```
  @@map("cancellations")
}

model DataSource {
  id          String    @id @default(cuid())
  name        String    @unique
  type        String
  description String?
  version     String?
  lastSyncAt  DateTime?
  recordCount Int       @default(0)
  status      String    @default("ACTIVE")
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  syncRuns    SyncRun[]

  @@map("data_sources")
}

model SyncRun {
  id             String     @id @default(cuid())
  dataSourceId   String
  version        String?
  source         String
  status         String     @default("SUCCESS")
  startedAt      DateTime   @default(now())
  completedAt    DateTime?
  insertedCount  Int        @default(0)
  updatedCount   Int        @default(0)
  rejectedCount  Int        @default(0)
  errorLog       String?

  dataSource     DataSource @relation(fields: [dataSourceId], references: [id],

  @@map("sync_runs")
}
```

Appendix B: Complete REST API

Endpoint Specification

Method	Endpoint Path	Auth Required	Purpose / Description
`POST`	`/api/auth/register`	Public	Register a new user account with Argon2id password hash.
`POST`	`/api/auth/login`	Public	Verify credentials, regenerate session, issue HTTP-only cookie.
`POST`	`/api/auth/logout`	Authenticated	Destroy session in database and clear client cookie.
`GET`	`/api/auth/me`	Authenticated	Retrieve profile details of currently authenticated user.
`GET`	`/api/stations`	Public	Search stations by code, name, city, or alias fragment.
`GET`	`/api/stations/:id`	Public	Lookup single station metadata by code or ID.
`GET`	`/api/search/trains`	Public	Point-to-point timetable search with availability and recovery.
`GET`	`/api/trains/:number`	Public	Retrieve scheduled train stops and coach layouts.
`GET`	`/api/fares/quote`	Public	Calculate authoritative server-side fare quotation.

Method	Endpoint Path	Auth Required	Purpose / Description
`POST`	`/api/bookings`	Authenticated	Create booking and atomically lock seat berths for 10 minutes.
`GET`	`/api/bookings`	Authenticated	Retrieve paginated personal booking history for logged-in user.
`GET`	`/api/bookings/:id`	Authenticated	Retrieve single booking by internal ID with ownership check.
`GET`	`/api/bookings/pnr/:pnr`	Authenticated	Retrieve single booking by 10-digit PNR with ownership check.
`POST`	`/api/bookings/:id/cancel`	Authenticated	Cancel confirmed booking and process tiered refund.
`POST`	`/api/payments/initialize`	Authenticated	Initialize payment transaction and obtain order reference.
`POST`	`/api/payments/:id/simulate-result`	Authenticated	Simulate payment outcome (`SUCCESS` / `FAILED`) and confirm seats.
`GET`	`/api/payments`	Authenticated	List payment transactions associated with user's bookings.
`GET`	`/api/tracking/:trainNumber`	Public	Fetch real-time train delay telemetry and

Method	Endpoint Path	Auth Required	Purpose / Description
			status.
`GET`	`/api/tracking/:trainNumber/route`	Public	Retrieve GeoJSON LineString coordinates for corridor route.
`GET`	`/api/tracking/stations/:code/live`	Public	Fetch real-time station live departure/arrival board.
`GET`	`/api/tracking/active-trains`	Public	Retrieve catalog of trackable train services.
`GET`	`/api/tracking/audit`	Public	Inspect health and latency audit of external telemetry providers.
`GET`	`/api/ai/status`	Public	Check Google Gemini service availability and model status.
`POST`	`/api/ai/chat`	Public	Submit conversational prompt to Gemini travel assistant.
`GET`	`/api/admin/summary`	Admin Only	Retrieve operational KPIs (revenue, bookings, trains, stations).
`GET`	`/api/admin/trains`	Admin Only	List all trains for master data management.
`POST`	`/api/admin/trains`	Admin Only	Create a new train service in master catalog.

Method	Endpoint Path	Auth Required	Purpose / Description
`PUT`	`/api/admin/trains/:id`	Admin Only	Update train service properties.
`DELETE`	`/api/admin/trains/:id`	Admin Only	Soft-deactivate a train service (`isActive = false`).
`GET`	`/api/admin/stations`	Admin Only	List all stations for master management.
`POST`	`/api/admin/stations`	Admin Only	Create a new station record in master database.
`PUT`	`/api/admin/stations/:id`	Admin Only	Update station details and coordinates.
`GET`	`/api/admin/routes`	Admin Only	List all route corridors and ordered intermediate stops.
`POST`	`/api/admin/routes`	Admin Only	Create a new route corridor.
`POST`	`/api/admin/route-stations`	Admin Only	Add an ordered intermediate stop to a route.
`POST`	`/api/admin/schedules`	Admin Only	Link a train to a route with departure time and running days.
`POST`	`/api/admin/fare-rules`	Admin Only	Define class tariff rates for a schedule.

Method	Endpoint Path	Auth Required	Purpose / Description
`GET`	`/api/admin/bookings`	Admin Only	Inspect all passenger bookings across system.
`GET`	`/api/admin/data-health`	Admin Only	View sync status, record counts, and dataset logs.
`GET`	`/api/health`	Public	System uptime, memory metrics, and database health probe.
`GET`	`/api/docs`	Public	Interactive Swagger UI API documentation.
`GET`	`/api/docs.json`	Public	Raw OpenAPI 3.0 specification in JSON format.

Table B.1: Complete REST API Route and Method Catalog.

Appendix C: Demonstration and Configuration Parameters

C.1 Demonstration Fare Tariff Parameters

Travel Class Code	Travel Class Name	Base Rate (Paise / 100 km)	Reservation Fee (Paise)	Superfast Charge (Paise)	Statutory GST (%)
`1A`	First Class AC	510 (₹5.10/km)	6,000 (₹60.00)	7,500 (₹75.00)	5%
`2A`	AC 2-Tier	300 (₹3.00/km)	5,000 (₹50.00)	4,500 (₹45.00)	5%
`3A`	AC 3-Tier	210 (₹2.10/km)	4,000 (₹40.00)	4,500 (₹45.00)	5%
`EXEC_CHAIR`	Executive Chair Car	330 (₹3.30/km)	5,000 (₹50.00)	6,000 (₹60.00)	5%
`CHAIR_CAR`	AC Chair Car	240 (₹2.40/km)	4,000 (₹40.00)	4,500 (₹45.00)	5%
`SLEEPER`	Sleeper Class (Non-AC)	70 (₹0.70/km)	2,000 (₹20.00)	3,000 (₹30.00)	0% (Exempt)
`SECOND`	Second Sitting	55 (₹0.55/km)	1,500 (₹15.00)	1,500 (₹15.00)	0% (Exempt)

Table C.1: Demonstration Fare Parameters by Travel Class.

C.2 Seed Master Network Corridor

The demonstration seed network configures an official 792 km Northern-to-Eastern transit corridor connecting New Delhi to Varanasi via Kanpur and Prayagraj:

Sequence	Station Code	Station Name	Zone	State	Distance from Origin (km)	Scheduled Dwell Time
1	`NDLS`	New Delhi	NR	Delhi	0 km	Origin (Dep: 06:00)
2	`CNB`	Kanpur Central	NCR	Uttar Pradesh	440 km	5 minutes
3	`PRYJ`	Prayagraj Junction	NCR	Uttar Pradesh	635 km	5 minutes
4	`BSB`	Varanasi Junction	NER	Uttar Pradesh	792 km	Terminus (Arr: 14:00)

- **Operating Train Service**: Train `22436` (Vande Bharat Express).
- **Configured Coaches**:
  - `C1`, `C2`, `C3`, `C4`: `CHAIR_CAR` (78 seats per coach, Window/Aisle layout).
  - `E1`: `EXEC_CHAIR` (52 seats per coach).
- **Default Running Calendar**: Daily operation (`runningDays = 127`).

Appendix D: Technical Glossary

- **ACID**: Set of database properties (Atomicity, Consistency, Isolation, Durability) guaranteeing reliable transaction processing.
- **Berth**: A dedicated sleeping position on a passenger train (categorized as Lower, Middle, Upper, Side Lower, or Side Upper).
- **CUID**: Collision-resistant Unique Identifier optimized for horizontal scaling and distributed database indexing.
- **Data Provenance**: Documentation of data lineage establishing whether information originated from live APIs, static open datasets, or simulated sandbox generators.
- **Double Booking**: A critical database defect where two conflicting passengers are concurrently assigned the same physical berth over intersecting journey segments.
- **Idempotency**: An API property ensuring that performing an identical request multiple times yields the exact same state without unintended side effects.
- **libSQL**: Open-source, standards-compliant fork of SQLite designed by Turso for edge deployment, client-server connections, and distributed replication.

- **Material Design 3 (M3)**: Google's design system prioritizing dynamic theming, structured visual elevation, and accessible contrast tokens.
- **O-D (Origin-Destination) Inventory**: An inventory management methodology where seat availability is evaluated across all constituent segment legs between a passenger's boarding and alighting points.
- **Paise**: The smallest monetary unit of Indian currency (1 Rupee = 100 paise). Used throughout RailFlow to guarantee integer arithmetic precision.
- **Passenger Name Record (PNR)**: A 10-digit unique record locator referencing a complete passenger reservation manifest and itinerary.
- **Prisma Driver Adapter**: Prisma ORM mechanism allowing custom database drivers (such as `@libsql/client`) to handle database transport layers.
- **Quota**: Segmented allocation pools reserved for specific demographic groups (General, Tatkal, Ladies, Senior Citizen).
- **Running Days Bitmask**: A 7-bit numerical value encoding the days of the week on which a train service operates.
- **Seat Hold**: A temporary 10-minute reservation lock during which passenger inventory is protected while payment settlement executes.
- **Serializable Isolation**: The highest ANSI SQL isolation level, ensuring that concurrent transactions produce identical results to serial execution.
- **Single-Page Application (SPA)**: A web application that loads a single HTML shell, dynamically rewriting view content via client-side routing.
- **Time-to-Live (TTL)**: A finite duration after which cached data or temporary reservation holds expire automatically.